

IMPLEMENTACIÓN Y EXTENSIÓN DE UN PROCESADOR RISC-V SOBRE CHISEL Y FPGA

**(Implementation and Extension of a RISC-V
processor using Chisel and FPGA)**

**Trabajo de Fin de Máster
para acceder al**

MÁSTER EN INGENIERÍA INFORMÁTICA

Autor: Noé Ruano Gutiérrez

Director: Pablo Prieto Torralbo

Septiembre - 2026

A mi padre y mi abuelo, maestros incesantes en el arte de vivir. A Mar, que ha sido una fuente inagotable de apoyo y comprensión, y a Pablo, mi director, por su enorme implicación para con este proyecto.

Resumen

RISC-V es una especificación de conjunto de instrucciones o ISA surgida en el año 2010 en la Universidad de Berkeley como una alternativa libre a los conjuntos de instrucciones (privativos) dominantes en aquel momento. Nacida de un proyecto académico destinado a facilitar la investigación en arquitecturas hardware, su modelo abierto y estandarizado, así como su filosofía, han favorecido su consolidación como una alternativa real a arquitecturas tradicionales como x86 o ARM en determinadas áreas.

Este documento pretende recoger el proceso de implementación de un núcleo RISC-V segmentado en cinco etapas, capaz de ejecutar el conjunto de instrucciones base definido en el estándar.

El proyecto tiene como punto de partida un diseño monociclo incompleto, desarrollado en la Universidad Nacional Cheng Kung (Taiwán). Esta implementación ha sido completada y estudiada en profundidad, quedando plasmadas en este documento las conclusiones de ese proceso de indagación.

Por último, se detalla el proceso de despliegue del diseño final sobre el chip FPGA de una placa de desarrollo usando el software Vivado.

Palabras clave: RISC-V, Chisel, HDL, arquitectura de computadores, diseño hardware.

Abstract

RISC-V is an open instruction set architecture (ISA) introduced in 2010 at the University of California, Berkeley, as a royalty-free alternative to the proprietary instruction sets that dominated the industry. It was originally conceived as an academic project to support research in computer architecture. Nevertheless, its open and standardised nature has contributed to its widespread adoption and has established it as a viable alternative to traditional architectures (such as x86 and ARM) in a variety of application domains.

This document aims to outline the process of implementing a five-stage pipelined RISC-V processor core, capable of executing the base instruction set defined by the standard.

The project is based on a single-cycle design developed at National Cheng Kung University (Taiwan). This design has been thoroughly analysed and serves as the foundation for the work presented herein.

Finally, the document describes the deployment of the resulting design on an FPGA development board using the Vivado design suite.

Palabras clave: RISC-V, Chisel, HDL, computer architecture, hardware design.

Índice general

1. Introducción	10
1.1. Motivación	10
1.2. Objetivos	11
1.3. Planificación y metodología seguidos	11
1.3.1. Formación previa	11
1.3.2. Construcción del computador monociclo	11
1.3.3. Construcción del procesador segmentado	11
1.3.4. Despliegue del diseño final sobre un FPGA	12
2. RISC-V	13
2.1. Historia de RISC-V	13
2.2. Especificaciones	14
2.3. El conjunto de instrucciones base RV32I	14
2.4. Formatos derivados	15
2.4.1. Formato B	15
2.4.2. Formato J	15
2.5. Extensiones	16
2.5.1. Extensión ‘M’	16
2.5.2. Extensión ‘Zicsr’	16
2.5.3. Extensión ‘F’	16
2.5.4. Extensión ‘Zifencei’	16
3. Herramientas y tecnologías	17
3.1. Chisel	17
3.1.1. Desarrollo en Chisel	17
3.1.2. Pruebas en Chisel	18
3.2. SBT	19
3.3. RISC-V GNU Toolchain	19
3.4. GTKWave	19
3.5. FPGA Arty A7-100T y AMD Vivado Design Suite	20
3.6. Git	21
4. El procesador monociclo	22
4.1. Descripción de la arquitectura de partida	22
4.1.1. TopModule	23
4.1.2. Memory	23
Señales de entrada	23
Señales de salida	23
4.1.3. InstructionROM y ROMLoader	24
4.1.4. CPU	25
Puertos de entrada	25

Puertos de salida	25
4.1.5. InstructionFetch	25
Señales de entrada	26
Señales de salida	26
4.1.6. InstructionDecode	26
Señales de entrada	26
Señales de salida	27
4.1.7. InstructionExecute	27
Señales de entrada	28
Señales de salida	28
4.1.8. MemoryAccess	28
Señales de entrada	28
Señales de salida	29
4.1.9. WriteBack	29
Señales de entrada	29
Señales de salida	29
5. El procesador segmentado	30
Dependencias de datos	31
Dependencias de control	31
5.1. Arquitectura de un registro de etapa	32
5.2. Segmentando el núcleo en dos etapas	33
5.3. Segmentando el resto del núcleo	37
5.3.1. Resolución de riesgos de datos (RAW)	37
Riesgos causados por la sucesión de instrucciones aritmético-lógicas	37
Dependencias en las que se encuentra involucrada la instrucción 'lw'	39
5.3.2. Resolución de riesgos de control	40
6. Despliegue de las implementaciones sobre FPGA	42
6.1. Proceso de despliegue	42
6.1.1. Obtención del HDL de bajo nivel	42
6.1.2. Configuración del fichero de <i>constraints</i>	44
Clase SlowTick	44
Clase BUFGCE	44
Modificaciones en el módulo Top	45
6.2. Análisis de los resultados temporales y de área obtenidos en Vivado	46
6.2.1. Análisis de la utilización	46
6.2.2. Análisis temporal	47
7. Conclusiones y trabajo futuro	48

Índice de figuras

2.1. Esquema con el formato que deben seguir las instrucciones del tipo B	15
2.2. Esquema con el formato que deben seguir las instrucciones del tipo J	15
3.1. Código y diagrama del multiplexor de 4 entradas en Chisel	18
3.2. Implementación de un test para el multiplexor de 4 entradas	19
3.3. Visualización de la traza generada tras la simulación del test	20
3.4. Placa de desarrollo Arty A7-100T	21
4.1. Diagrama de bloques de la arquitectura global del computador	22
4.2. Implementación de la clase RAMBundle	24
4.3. Acoplamiento de la memoria en función del estado de la carga del programa.	24
4.4. Conexión del contador de programa a la salida del multiplexor que determina su valor a cada ciclo en función de los saltos que se producen en el flujo de ejecución de los programas cargados en el computador	26
4.5. Generación de las señales de habilitación de lectura/escritura en memoria	27
5.1. Ejecución de un programa en un computador con diseño monociclo (a) y segmentado (b)	30
5.2. Ejemplos de riesgos de datos. (a) RAW (<i>Read After Write</i>): lectura tras escritura. (b) WAW (<i>Write After Write</i>): escritura tras escritura. (c) WAR (<i>Write After Read</i>): escritura tras lectura	31
5.3. Flujo de ejecución de dos instrucciones implicadas en una dependencia RAW. Se señala en color verde el instante a partir del cual la segunda instrucción podrá leer el dato actualizado en el registro <code>x1</code> (ciclo 5).	31
5.4. Representación del flujo de ejecución de un programa sobre una arquitectura segmentada sin mecanismos para resolver riesgos de control.	32
5.5. Implementación inicial del registro de etapa que separa las fases de <i>fetch</i> y decodificación.	32
5.6. Representación del flujo de ejecución de un programa sobre una arquitectura segmentada en dos etapas sin mecanismos para resolver riesgos de control.	33
5.7. Desensamblado, código ensamblador y pseudocódigo de un programa sencillo.	33
5.8. Código del test con el que se pone a prueba el funcionamiento de un programa sencillo sobre el núcleo segmentado en dos fases.	34
5.9. Sección de la traza obtenida tras la ejecución del test mostrado en la Figura 5.7	35
5.10. Misma sección de la traza pero con una instancia de la clase <code>Mem</code> como entidad de memoria en lugar de <code>SyncReadMem</code>	36
5.11. Sección de la traza en la que se muestra la pronta ejecución del bucle final.	36
5.12. Comparación entre la implementación original del módulo <code>InstructionFetch</code> y su adaptación para resolver riesgos de control.	36
5.13. Ejemplo de una dependencia RAW en una arquitectura segmentada en cinco fases.	37
5.14. Implementación en Chisel y diagrama de decisión de la lógica de <i>forwarding</i> para la selección del primer operando de la ALU.	38

5.15. Flujo de ejecución de dos instrucciones implicadas en una dependencia de datos de tipo RAW, una de las cuales es la instrucción <code>lw</code>	39
5.16. Cómputo de la señal <code>lw_stall</code>	40
5.17. Instanciación, en el módulo CPU, del registro de segmentación que separa las fases de decodificación y ejecución.	40
5.18. Adaptación del código del módulo <code>InstructionFetch</code>	40
5.19. Diagrama ilustrativo del proceso de mitigación de los riesgos de control.	41
5.20. Configuración de la señal de borrado de los registros de segmentación que separan las fases de fetch-decodificación y decodificación-ejecución, conforme al valor del flag de salto en fase de ejecución.	41
6.1. Diagrama de bloques con los módulos de los que se compone el computador.	42
6.2. Implementación de la CPU completa, con la definición del punto de entrada a la función que genera el circuito en lenguaje Verilog.	43
6.3. <i>Constraints</i> configurados para probar el diseño final en la placa.	44
6.4. Implementación de los módulos <code>SlowTick</code> y <code>BUFGCE</code> , y modificaciones realizadas en el módulo <code>Top</code>	45
6.5. Comparativa de los resultados de utilización obtenidos.	46
6.6. Selección de algunos de los valores obtenidos tras análisis temporal.	47

1 Introducción

1.1. Motivación

En la actualidad existen principalmente dos modelos de negocio en el ámbito del diseño y/o fabricación de procesadores. Por un lado estarían aquellas compañías cuyo beneficio proviene principalmente de la venta de licencias para el uso de sus diseños, como pudiera ser el caso de ARM, al tiempo que existen otras las cuales centran su actividad empresarial tanto en el diseño como en la fabricación y venta de hardware (caso de Intel), sin perjuicio de que pudieran obtener beneficios también del pago por el uso de su propiedad intelectual con fines comerciales [1].

Ambos modelos se basan en la siguiente premisa: que la especificación del conjunto de instrucciones que deberá ejecutar un computador constituye por sí mismo una entidad patentable. Esto obliga a todo aquel que quisiera obtener beneficio de la comercialización de productos derivados de esas especificaciones a pagar regalías al tenedor de la propiedad intelectual, lo cual obstaculiza de manera notable el surgimiento de innovaciones que pudieran aprovechar las ya existentes en materia de conjuntos de instrucciones y arquitecturas de largo recorrido. Ejemplos de estas arquitecturas son x86-64 o ARM, cuyo éxito ha quedado demostrado históricamente.

No obstante, en los últimos años del siglo XX y principios del XXI surgieron corrientes que apostaban por la creación de arquitecturas libres con proyectos como Sparc u OpenRISC, este último aún en desarrollo [2]. Los beneficios del hardware libre son innegables, ya que permite experimentar con arquitecturas personalizadas sin restricciones comerciales, facilitando la educación y la investigación, agilizando los procesos de creación y prueba de prototipos, y contribuyendo, en definitiva, a democratizar el acceso a la tecnología.

No sería hasta el año 2011 [3] cuando en la Universidad de Berkeley comenzaría a gestarse el que es sin lugar a dudas el ISA abierto que mayor renombre ha conseguido tomar hasta la fecha: RISC-V. Tanto es así que gigantes tecnológicos como Google, Huawei, Microsoft o Nvidia [4] realizan importantes aportaciones monetarias de manera regular a la fundación que mantiene y desarrolla la especificación (RISC-V International).

Esta fundación recibe, además, las contribuciones de multitud de instituciones públicas. Por ejemplo, en el ámbito europeo, la EPI¹ es la iniciativa mediante la cual se canalizan los esfuerzos de la academia e industria europeas por hacer realidad el proyecto de una arquitectura abierta, robusta y capaz, que sitúe a la Unión Europea a la vanguardia de la computación de altas prestaciones. Es indudable el aporte de esta iniciativa a la soberanía tecnológica y digital de Europa donde, recientemente, se ha aprobado el *Chips Act 2.0*[5]: una propuesta en la que se recogen medidas para impulsar la industria de los microprocesadores, apoyando la producción dentro de la Unión Europea y reduciendo la dependencia de agentes externos.

En conclusión, el principal motivo que ha impulsado el desarrollo de este trabajo ha sido la posibilidad de conocer en profundidad y trabajar con una de las tecnologías clave para el futuro tecnológico de la Unión Europea.

¹European Processor Initiative

1.2. Objetivos

El principal objetivo del proyecto es la implementación de un computador sencillo, mononúcleo, segmentado y en orden, que pueda ejecutar el conjunto de instrucciones definido en la especificación del ISA abierto RISC-V, en su versión de 32 bits. Para ello se partirá de un diseño ya existente, monociclo e incompleto, elaborado por el investigador de la Universidad Nacional Cheng Kung, Ching-Chun (Jim) Huang, quien lo emplea como material docente en una asignatura de arquitectura y tecnología de computadores impartida en esa misma universidad.

El diseño de partida se encuentra descrito en el lenguaje Chisel, que es un lenguaje de propósito específico o DSL (Domain-Specific Language) construido sobre el lenguaje de programación Scala, este último, un lenguaje de propósito general o GPL (General Purpose Language). Esto requerirá las adquisición, por parte del autor del presente trabajo, del conocimiento sobre ambos lenguajes que le permita, a posteriori, afrontar la tarea de completar el pipeline del mencionado diseño original para, más adelante, modificarlo de manera que este se encuentre segmentado en cinco etapas.

La segmentación del diseño original implicará, en primer lugar, un estudio profundo de la estructura interna del procesador, analizando el comportamiento y el conexionado de los diferentes bloques en los que se encuentra organizada la arquitectura del núcleo para, más adelante, integrar en él los registros de etapa y la unidad de riesgos con los que se transformará la arquitectura monociclo en una arquitectura en pipeline.

Durante todo el proceso se hará uso de las herramientas descritas más adelante en el Capítulo 3, las cuales posibilitarán la identificación de los errores cometidos en el proceso de segmentación de la arquitectura.

Por último, se pretende elaborar una prueba de concepto consistente en el despliegue del diseño final sobre una placa FPGA (Field Programmable Gate Array), de modo que pueda comprobarse sobre hardware real el buen funcionamiento y desempeño de este diseño.

1.3. Planificación y metodología seguidos

1.3.1. Formación previa

En primer lugar se cursará, de manera autónoma, una formación relativa a los lenguajes Scala y Chisel². Esta formación fue creada por diversos investigadores y colaboradores de la Universidad de California, Berkeley, y consiste en un cuaderno de Jupyter en el que se abordan, inicialmente, aspectos introductorios relativos al desarrollo de software en el lenguaje Scala para, posteriormente, proporcionar al usuario nociones fundamentales sobre el diseño de hardware empleando el lenguaje Chisel en lecciones de complejidad incremental.

1.3.2. Construcción del computador monociclo

La segunda fase del proyecto consistirá en la implementación de las partes incompletas en la arquitectura de partida. Ello implicará la puesta en práctica y consiguiente asentamiento de los conocimientos adquiridos durante la realización del bootcamp sobre Scala y Chisel, además de que constituirá en sí mismo un proceso de descubrimiento de la arquitectura del núcleo monociclo y las particularidades de su implementación.

1.3.3. Construcción del procesador segmentado

A continuación, con el diseño base completado, se procederá a segmentar el núcleo en las cinco etapas siguientes:

1. Lectura de instrucción o fase de *fetch*

²Disponible en: www.github.com/freechipsproject/chisel-bootcamp

2. Decodificación de la instrucción
3. Ejecución
4. Lectura/escritura de resultados en memoria
5. Escritura en el banco de registros, o fase de *writeback*

La integración de los registros de etapa requerirá un análisis minucioso del conexionado interno del procesador, e implicará la alternancia continua de fases de desarrollo (construcción y adhesión de los registros al pipeline) y fases de prueba hasta verificar que la ejecución de los binarios cargados desemboca en los resultados esperados a nivel del estado arquitectónico.

1.3.4. Despliegue del diseño final sobre un FPGA

Para concluir el proyecto se elaboró una prueba de concepto consistente en el despliegue del diseño elaborado sobre una placa FPGA. Para ello se implementó, también en lenguaje Chisel, una interfaz con la que poder comunicar el computador con los dispositivos presentes en la placa. Ello ha posibilitado el envío (por medio de un módulo UART), la carga en memoria y la ejecución de binarios compilados para la arquitectura RISC-V.

2 RISC-V

RISC son las siglas de ‘Reduced Instruction Set Computer’, es decir, un computador con un conjunto de instrucciones reducido. Una de las primeras menciones de este concepto la encontramos en la década de 1970. En aquel entonces, en IBM, tenían la necesidad de desarrollar un computador sencillo, pero capaz de ejecutar 12 millones de instrucciones por segundo, de modo que pudiera ser integrado en un sistema de enrutamiento de llamadas telefónicas [6].

En ese periodo muchas arquitecturas comerciales seguían el paradigma CISC (Complex Instruction Set Computer), caracterizado por conjuntos de instrucciones amplios y relativamente complejos. Con frecuencia, esas instrucciones deben ser interpretadas a nivel arquitectural y traducidas a un microcódigo con el que se ejecutaban las operaciones necesarias para obtener los resultados esperados. No obstante, este enfoque requiere contar, en primer lugar, con un intérprete de bajo nivel que traduzca las instrucciones en sus correspondientes microcódigos, además de una memoria dedicada en la que alojarlos.

Las arquitecturas RISC surgen como respuesta a la tendencia de los conjuntos de instrucciones a ganar complejidad con el tiempo, a pesar de que algunos estudios [7] apuntaban que la mayoría de los programas desarrollados por aquel entonces tan solo empleaban un subconjunto de las instrucciones disponibles. En estos estudios se proponía la creación de arquitecturas más sencillas, baratas de producir y eficientes, las cuales, aún siendo sencillas, fueran capaces de ejecutar programas complejos de igual modo que lo hacían las arquitecturas CISC.

Con el tiempo, numerosos principios asociados al diseño RISC han sido adoptados de manera generalizada en las arquitecturas de procesadores modernos. De hecho, incluso aquellas tradicionalmente clasificadas como CISC emplean internamente representaciones simplificadas de sus instrucciones, lo que facilita su ejecución en pipelines profundos y en arquitecturas con ejecución fuera de orden. En consecuencia, la distinción clásica entre RISC y CISC ha tendido a difuminarse.

En los últimos años, el interés por arquitecturas basadas en estos principios se ha intensificado con la aparición de arquitecturas abiertas que permiten estudiar, modificar e implementar procesadores completos sin las limitaciones impuestas por especificaciones propietarias. En este contexto, RISC-V se fundamenta en los principios clásicos del diseño RISC, caracterizados por la simplicidad del conjunto de instrucciones, la regularidad de su formato y la eficiencia en la ejecución. A estos elementos añade un modelo de especificación abierto que permite su implementación, modificación y extensión sin costes de licencia ni restricciones adicionales, facilitando la experimentación y la personalización de la arquitectura y fomentando la colaboración entre los ámbitos académico e industrial. Como resultado, la combinación de simplicidad arquitectónica y accesibilidad ha impulsado una rápida adopción de RISC-V en diversos contextos, consolidándola como una plataforma especialmente idónea tanto para la docencia como para la investigación en arquitectura de computadores.

2.1. Historia de RISC-V

El manual de la primera versión del ISA abierto RISC-V fue publicado en el año 2011[8]. Por aquel entonces, los investigadores de la Universidad de California, Berkeley, Krste Asanović, Yunsup Lee y Andrew Waterman, bajo la dirección de David A. Patterson, integraban el equipo *ParLab*, trabajando en un proyecto de investigación sobre computación paralela, financiado por Intel y Microsoft, y del cual surgieron como derivados del trabajo de investigación tanto el ISA RISC-V, como el lenguaje de

construcción de hardware (HDL) Chisel [9].

Más tarde, en el año 2015, se creó la RISC-V Foundation con el objetivo de organizar los esfuerzos de la comunidad global de desarrolladores de software y hardware, por impulsar la adopción del estándar, mantenerlo y hacerlo evolucionar de una manera organizada y eficaz. Además, en este mismo año, los integrantes del *ParLab* (a excepción de Patterson) fundarían SiFive, una compañía dedicada al diseño y venta (que no fabricación) de procesadores basados en RISC-V.

2.2. Especificaciones

La especificación se organiza principalmente en dos volúmenes: la especificación del conjunto de instrucciones no privilegiado y la especificación del conjunto privilegiado, siendo el primero aquel sobre el cual se ha puesto el foco a la hora de proponer y desarrollar este trabajo. La especificación del ISA privilegiado recoge las instrucciones y modos de ejecución necesarios para ejecutar sistemas operativos y operar con periféricos [10], mientras que la especificación del ISA no privilegiado recoge las instrucciones necesarias para la construcción de arquitecturas RISC-V básicas pero completamente funcionales.

El ISA no privilegiado lo componen, a su vez, cuatro especificaciones básicas junto con sus respectivas extensiones. Esas especificaciones básicas deben estar presentes en toda implementación que quiera hacerse del ISA, es decir, que una implementación debería integrar una de las cuatro especificaciones base, junto con tantas extensiones como se requiera. Las especificaciones base recogen el conjunto de instrucciones básico que todo núcleo RISC-V debería ser capaz de ejecutar [11], y son las siguientes:

- **RV{32,64}I**: ISAs con un XLEN (longitud de registro) de 32 y 64 bits.
- **RV{32,64}E**: subconjuntos de RV32I y RV64I con la mitad de registros. Elaborados para propiciar la construcción de microcontroladores de tamaño reducido.

Las instrucciones de las especificaciones base posibilitan la ejecución de operaciones aritmético-lógicas (suma, resta, operaciones lógicas, *lui*¹ y *auipc*²), saltos condicionales y no condicionales, así como operaciones de lectura y escritura en memoria [12]. En el ISA base se definen, además, las operaciones de ordenación de memoria necesarias para garantizar la consistencia en sistemas con múltiples hilos de ejecución paralelos o *harts*³ [13].

Asimismo, se definen las instrucciones ‘ECALL’ y ‘EBREAK’ cuya utilidad es, respectivamente, realizar llamadas al sistema y pausar la ejecución de un *hart* para inspeccionar el estado del banco de registros y la memoria (útil en labores de depuración) [14].

Por último, la especificación indica el formato de otro tipo de instrucciones las cuales, aún siendo válidas, pueden no tener ningún efecto sobre el estado de la arquitectura. Estas instrucciones, denominadas ‘HINTs’, se reservan para casos en que se desee agregar una cierta funcionalidad al ISA, la cual no tenga por qué ser adoptada obligatoriamente por una arquitectura ya existente, pudiendo esta simplemente ignorar la instrucción [15].

2.3. El conjunto de instrucciones base RV32I

En el manual de la arquitectura se indica que en el conjunto RV32I las instrucciones deben poder direccionar a 32 registros de propósito general. No obstante, tan solo se restringe el uso del registro ‘x0’ para albergar una constante de valor 0, quedando recogido el uso del resto de registros en la ABI⁴ de RISC-V [16].

¹*load upper immediate*: carga en un registro un entero de 32 bits construido con los 20 bits más significativos de la instrucción (inmediato), a los cuales se aplica un shift lógico de 12 bits hacia la izda.

²*add upper immediate (to) program counter*: construye un inmediato de 32 bits exactamente igual que *lui*, lo suma al valor del contador de programa y lo guarda en un registro

³*hardware thread*: una unidad de ejecución hardware capaz de ejecutar de forma independiente un flujo de instrucciones. En RISC-V, cada hart dispone de su propio contador de programa y de su estado arquitectónico.

⁴*Application Binary Interface*: una especificación de la forma en que los programas deben ser llamados a nivel de lenguaje máquina (paso de parámetros, retorno de resultados, etc.), así como la forma en que deben construirse y enlazarse los ejecutables que los albergan

En lo que respecta al formato de las instrucciones, existen cuatro formatos base [17]:

- **R**: operaciones aritmético-lógicas en las que los dos operandos son registros.
- **I**: operaciones aritmético-lógicas en las que uno de los operandos es un inmediato codificado en la propia instrucción. Este formato es el empleado, además, en la codificación de la instrucción de lectura de memoria *lw* y sus derivados, así como en la codificación de la instrucción de salto incondicional *jalu*⁵.
- **S**: instrucción de escritura en memoria *sw* y sus derivados.
- **U**: es el formato empleado en la codificación de las instrucciones *lui* y *auipc*.

2.4. Formatos derivados

2.4.1. Formato B

El formato B se emplea en la codificación de las instrucciones de salto condicional. Sobre este formato merece especial atención la forma en que se construye e interpreta el inmediato.

Este valor representa, en bytes, la distancia que separa a la instrucción de salto de una instrucción objetivo en memoria. Este offset siempre va a ser un múltiplo de 2, por lo que el último dígito binario del inmediato puede asumirse siempre nulo. Teniendo esto en cuenta, el inmediato en las instrucciones de salto condicional codifica los 12 bits más significativos que preceden a ese último dígito binario.

Para formar el offset, se añadiría un bit nulo en la posición menos significativa del inmediato, y este se extendería hasta 32 bits de tal modo que pueda ser sumado al contador de programa para formar la dirección efectiva del salto.

A continuación, en la Figura 2.1, se presenta un diagrama con la estructura de una instrucción de salto condicional.

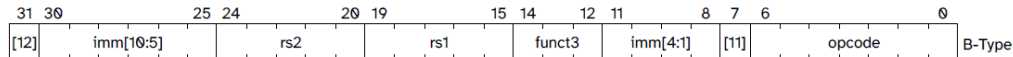


Figura 2.1: Esquema con el formato que deben seguir las instrucciones del tipo B

2.4.2. Formato J

El formato J debe emplearse en la codificación de las instrucciones de salto incondicional. Se trata de una variación del tipo U, que sigue la lógica del formato B en cuanto a la construcción del inmediato, tal y como puede observarse en la Figura 2.2 el offset se encuentra codificado en múltiplos de 2 por lo que el último bit va a ser siempre 0, con lo que puede codificarse dentro de la instrucción los 19 bits más significativos inmediatamente posteriores a ese último dígito binario.

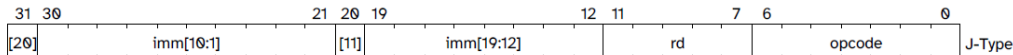


Figura 2.2: Esquema con el formato que deben seguir las instrucciones del tipo J

⁵*jump and link register*: avanza el contador de programa a una posición igual a su valor actual más el offset formado con la extensión de signo del inmediato a 32 bits. Además, guarda la dirección de retorno (instrucción siguiente al salto) en un registro de destino codificado en la instrucción

2.5. Extensiones

A continuación se presentan algunas de las principales extensiones contempladas en la especificación de RISC-V, diseñadas para ampliar las funcionalidades del conjunto base de instrucciones y permitir su adaptación a diferentes dominios de aplicación.

2.5.1. Extensión ‘M’

Para simplificar la implementación del conjunto de instrucciones base se decidió incorporar las instrucciones de multiplicación y división de enteros a su propia extensión en la que se definen, de manera separada, el formato de las instrucciones de multiplicación (MUL, MULHU y MULHSU⁶) y el de las instrucciones de división (DIV⁷, DIVU⁸ y REM(U)⁹).

2.5.2. Extensión ‘Zicsr’

La especificación del conjunto de instrucciones base no contempla la presencia de un registro de estado y control o ‘CSR’¹⁰. No obstante a ello, en la especificación del ISA no privilegiado sí se definen un conjunto de operaciones que son consideradas de utilidad en sistemas que gestionen múltiples *harts*. Estas operaciones incluyen el manejo de contadores y temporizadores, así como la notificación del estado de las operaciones de punto flotante [18].

2.5.3. Extensión ‘F’

La extensión ‘F’ recoge las operaciones de punto flotante en precisión simple, y define sus formatos y comportamiento, así como la presencia de un conjunto de 32 registros dedicados, además de un registro de estado propio (*fcsr*) y la disposición de sus campos.

2.5.4. Extensión ‘Zifencei’

En esta extensión se detallan el formato y comportamiento de la instrucción *FENCE.I*, empleada en la sincronización entre escrituras sobre la memoria de instrucciones, y las lecturas sobre esa misma memoria [19]. Esta instrucción toma especial relevancia en contextos donde el código de un *hart* que ejecuta en un sistema con arquitectura Harvard¹¹, puede hacer que se sobrescriba a sí mismo. Tras una escritura en un área de la memoria correspondiente al código del programa, este podría invocar la instrucción *FENCE.I* para garantizar la coherencia en la memoria de instrucciones con respecto a la de datos, que es donde realiza la escritura. La primera, en el momento de ejecutar la instrucción escrita previamente, podría leer en la dirección de la instrucción un dato inconsistente, en función de si alguna lectura de instrucciones anteriores hubiera forzado o no la actualización de esa posición en la memoria.

⁶Las instrucciones MULHU y MULHSU retornan los *XLEN* bits más significativos en resultados cuya representación requiera de, a lo sumo, $2 * XLEN$ bits

⁷Cociente con signo

⁸Cociente sin signo

⁹Resto de los cocientes con y sin signo

¹⁰Control-Status Register

¹¹Arquitecturas donde la memoria se encuentra dividida, proporcionando buses dedicados e independientes para acceder a la memoria de datos y a la de instrucciones

3 Herramientas y tecnologías

3.1. Chisel

Chisel es un lenguaje de descripción de hardware de alto nivel gestado en el año 2012 en el seno del Departamento de Ingeniería Eléctrica y Ciencias de la Computación (abrev. EECS) de la Universidad de California, Berkeley. Este lenguaje surgió como parte de un proyecto de investigación cuyo objetivo final era la implementación de una interfaz de lenguaje que proporcionase un nivel de abstracción superior al ofrecido por HDL's convencionales como son VHDL o Verilog. Además, se desarrollaría un conjunto de herramientas software con las que traducir los diseños elaborados en Chisel, en sus equivalentes VHDL o Verilog. De este modo, podrían ejecutarse los procesos de síntesis o emulación con los que evaluar el funcionamiento de los módulos desarrollados [20].

El hecho de que se trate de un HDL de alto nivel tiene un impacto directo y positivo sobre la celeridad en los procesos de desarrollo y prueba del hardware, permitiendo elaborar y probar diseños de una manera ágil sin perder las características de eficiencia y robustez que proporcionan lenguajes de más bajo nivel.

Es un lenguaje de propósito específico construido sobre Scala, un lenguaje de propósito general orientado a objetos, siendo esta la principal propiedad que permite a Chisel mostrar al usuario una interfaz de diseño de hardware de tan alto nivel y, por ende, amigable tanto para el desarrollador novel como para usuarios más avanzados, y es que resulta muy práctico el poder modelar un circuito lógico como un conjunto de clases relacionadas entre sí. Cada una de estas clases define el comportamiento interno de un componente específico, mientras que las relaciones entre las distintas clases que conforman el diseño global representan las conexiones físicas entre los módulos del sistema. Este enfoque no solo facilita la estructuración del diseño, sino que también permite trasladar al desarrollo hardware principios propios de la ingeniería del software tales como la abstracción, la modularidad y la reutilización de código, favoreciendo así la construcción de sistemas complejos a partir de componentes bien definidos y fácilmente mantenibles.

3.1.1. Desarrollo en Chisel

La unidad fundamental de diseño de hardware en Chisel son los módulos, que encapsulan la disposición interna y el comportamiento de un circuito lógico. Un módulo puede componerse a partir de la instanciación de otros módulos, cables internos o cables de entrada/salida, siendo empleados estos últimos en la evaluación de las expresiones lógicas que definirán el comportamiento del módulo. Este comportamiento puede describirse por medio de operadores aritmético-lógicos convencionales, o bien, por medio de los tipos de datos y componentes definidos en las librerías de Chisel, tales como multiplexores, decodificadores o incluso memorias.

A continuación, en la Figura 3.1, se proporciona un código sencillo con el que se describe un multiplexor de 4 entradas de 32 bits cada una.

Nótese que los módulos se definen como clases que extienden la clase *Module*, la cual dota a los módulos de una plantilla con los atributos y métodos esenciales para su construcción, siendo *IO* uno de esos métodos. Este método se emplea en la construcción de la interfaz de entrada-salida de los módulos, y el modo de definir esta interfaz es tal y como se aprecia en las líneas 6 a 14 del ejemplo en la Figura 3.1: el método *IO* recibe como parámetro una instancia de la clase *Bundle*, dentro de la cual se definen

los puertos de entrada y salida del componente.

Sobre la clase *Bundle*, el propósito de esta es posibilitar la creación de nuevos tipos de datos al estilo de los *struct* en lenguaje C.

```
1 package mux4
2 import chisel3._
3 import chisel3.util._
4
5 class Mux4 extends Module {
6   val io = IO(new Bundle{
7     val i1 = Input(UInt(32.W))
8     val i2 = Input(UInt(32.W))
9     val i3 = Input(UInt(32.W))
10    val i4 = Input(UInt(32.W))
11    val sel = Input(UInt(2.W))
12
13    val o = Output(UInt(32.W))
14  })
15
16  io.o := MuxLookup(io.sel, io.i1)(
17    Seq(
18      0.U -> io.i1,
19      1.U -> io.i2,
20      2.U -> io.i3,
21      3.U -> io.i4,
22    ))
23 }
```

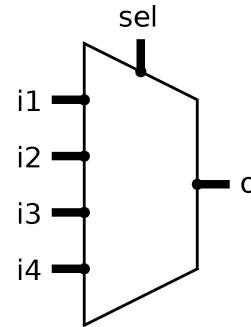


Figura 3.1: Código y diagrama del multiplexor de 4 entradas en Chisel

3.1.2. Pruebas en Chisel

Como se comentaba al comienzo de esta sección sobre Chisel, una de sus características más reseñables es la agilidad con la que puede probarse el comportamiento de los módulos, y ello es gracias a los tests.

Un test de Chisel es una clase que permite simular el comportamiento de un módulo y verificar su funcionamiento mediante la aplicación de estímulos y la comprobación de las respuestas generadas por la lógica del módulo.

Se ha decidido utilizar *ChiselScalatestTester* como interfaz de pruebas en lugar de la más reciente *ChiselSim*. Dado que no se aprecian diferencias relevantes a nivel de implementación entre ambas interfaces, la elección se ha fundamentado en la equivalencia funcional entre ellas y en la necesidad de reutilizar el conjunto de tests ya desarrollados, evitando así esfuerzos de adaptación innecesarios.

Las interfaces de prueba de Chisel definen cinco funciones básicas con las que evaluar el comportamiento de un módulo: *peek*, *poke*, *expect*, *step* y *stepUntil*. Estas funciones se invocan sobre las diferentes señales de las cuales se componga el módulo y permiten realizar las siguientes acciones:

- ***peek***: lectura del valor de una señal en el instante de invocación de la función.
- ***poke***: cambio en el valor de la señal.
- ***expect***: evaluación de un valor determinado en la señal, esto es, probar si el valor de la señal en el instante de invocación de la función es el esperado.
- ***step***: se invoca sobre una señal de reloj y permite avanzarlo un ciclo.
- ***stepUntil***: avanza el reloj hasta que se cumpla una condición en alguna otra señal.

En la Figura 3.2 se muestra un ejemplo de test que evalúa el comportamiento del módulo definido en el código de la Figura 3.1. En el código del test se establece el valor de los cuatro puertos de entrada por medio del método *poke* y, luego, se itera con los posibles valores de la señal de selección, comprobando, por medio del método *expect*, que el valor en el puerto de salida es el esperado.

```

1 package mux4
2 import chisel3._
3 import chiseltest._
4 import org.scalatest.flatspec.AnyFlatSpec
5 import mux4.Mux4
6
7 class Mux4Test extends AnyFlatSpec with ChiselScalatestTester {
8   behavior.of("Mux4")
9   it should "output the right input, given any valid selector value" in {
10     test(new Mux4) { c =>
11       c.io.i1.poke(1.U)
12       c.io.i2.poke(2.U)
13       c.io.i3.poke(3.U)
14       c.io.i4.poke(4.U)
15       var x = 0
16       for (x <- 0 to 3) {
17         c.io.sel.poke(x.U)
18         c.io.o.expect(x + 1)
19       }
20     }
21   }
22 }

```

Figura 3.2: Implementación de un test para el multiplexor de 4 entradas

3.2. SBT

SBT es una herramienta de construcción de software semejante a Maven o Gradle, y es la más utilizada por los desarrolladores en lenguaje Scala[21]. Su uso es por medio de una interfaz de línea de comandos, proporcionando una consola interactiva desde la que pueden realizarse todo tipo de tareas administrativas sobre los proyectos, tales como lanzar pruebas, ejecutar un programa principal, compilar ficheros de código fuente de manera individual o construir el proyecto, requiriendo esto último la creación de un fichero de configuración *build.sbt* dentro del directorio del proyecto.

Escrito también en el lenguaje Scala, en él se definirán las propiedades del proyecto, como puedan ser su nombre y versión, la versión de Scala empleada en la implementación o las librerías de las que depende, siendo estas dos últimas propiedades imprescindibles de cara al proceso de compilación.

3.3. RISC-V GNU Toolchain

Gran parte de las pruebas llevadas a cabo sobre la implementación del núcleo han consistido en la carga, ejecución y depuración de programas desarrollados en lenguaje C, lo cual ha implicado aplicar un proceso de compilación cruzada para obtener binarios que pudieran ejecutar sobre la arquitectura RISC-V.

Este proceso ha sido llevado a cabo por medio de la suite de herramientas de que se compone el toolchain de GNU para RISC-V[22], disponible en sistemas Linux basados en Debian por medio del gestor de paquetes *apt* (paquete *gcc-riscv64-unknown-elf*).

3.4. GTKWave

La depuración de componentes arquitecturales individuales tales como la ALU o el banco de registros, o incluso fases completas dentro del pipeline, es una tarea que puede acometerse de manera ágil por medio de tests de Chisel, estableciendo primeramente los casos de prueba a implementar, y verificando que los cambios en las señales bajo supervisión son los esperados para el conjunto de entradas definido en cada

caso de prueba. No obstante, este proceso de depuración del pipeline se torna más complejo cuando se desea verificar el comportamiento global de la arquitectura mediante la ejecución de un programa, lo cual requiere la implementación de tests que verifiquen el estado de un número mucho mayor de señales a cada ciclo, incrementando notablemente la complejidad de los tests desarrollados y del proceso de depuración.

Aún siendo esta una tarea ciertamente abordable, se optó por explorar alternativas que permitiesen depurar el estado de la arquitectura de una manera más rápida y visual, y se consideró que la herramienta GTKWave cumpliría perfectamente con este propósito.

GTKWave es una herramienta de análisis empleada en la depuración de trazas generadas en los procesos de simulación de modelos elaborados con Verilog o VHDL. Estas trazas VCD, generadas con la ejecución de los tests de Chisel, se almacenan en ficheros con un formato definido originalmente en el estándar IEEE 1364-1995[23] y constituyen una descripción de todos los cambios experimentados por las señales de los módulos durante la ejecución de las simulaciones.

La herramienta permite explorar esos cambios de una manera visual por medio de una interfaz gráfica en la que puede explorarse el histórico de cambios de la totalidad de las señales que conformen cada módulo, tal y como se muestra en la Figura 3.3, donde se presenta una captura de pantalla de la interfaz gráfica de GTKWave, mostrando una visualización de la traza generada tras la ejecución del test mostrado en la Figura 3.2.

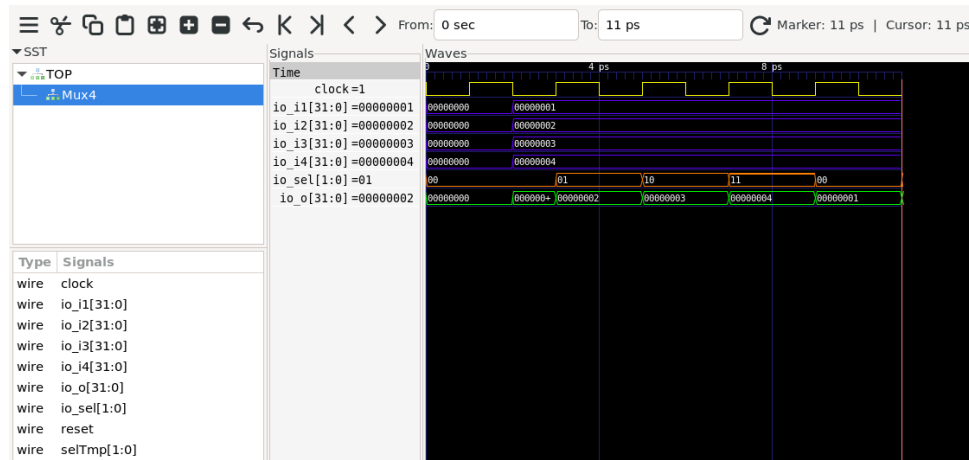


Figura 3.3: Visualización de la traza generada tras la simulación del test

3.5. FPGA Arty A7-100T y AMD Vivado Design Suite

AMD Vivado es un software de diseño hardware para SoC¹ y FPGAs desarrollado y mantenido por la empresa AMD que es, además, el fabricante de la placa de desarrollo Artit A7-100T empleada en las pruebas llevadas a cabo como parte del proceso de verificación final de la arquitectura diseñada.

La placa alberga el FPGA AMD (anteriormente Xilinx) Artix7, con 101440 celdas lógicas[24], superficie más que suficiente para albergar el núcleo segmentado. Además, tal y como puede verse en la imagen de la placa presentada en la Figura 3.5², esta cuenta con numerosos elementos de que serán usados para las distintas pruebas de verificación. Se incluyen switches de pulsación (10) y lineales (9), diodos LED (números 7 y 8), interfaces Ethernet (3) y UART (bajo USB; número 2), o conectores para módulos de expansión de Diligent (*pmod*³; número 16) y shields de Arduino (11).

¹System-on-Chip: circuitos integrados que incorporan todos o la gran mayoría de los componentes que constituyen un computador

²Diligent. (2023). Arty A7 - Diligent Reference [png]. Arty A7 Reference Manual. Disponible aquí

³*Peripheral module interface*

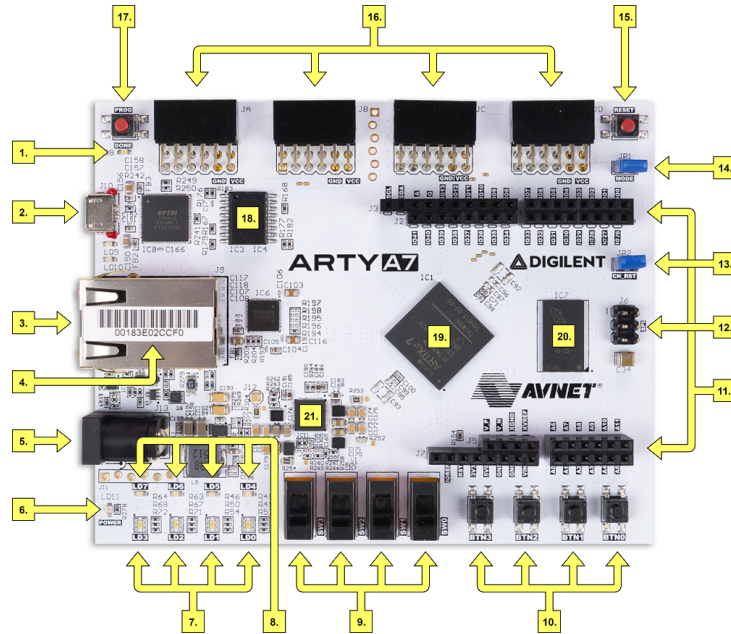


Figura 3.4: Placa de desarrollo Arty A7-100T

3.6. Git

Git fue concebido originalmente por Linus Torvalds como una alternativa de código abierto a BitKeeper, el software de control de versiones propietario empleado en el desarrollo del kernel de Linux entre los años 2002 y 2005[25], y es actualmente el software de control de versiones más utilizado por los desarrolladores, según los datos reflejados en el *Stack Overflow Annual Developer Survey* del año 2022[26]

El repositorio de este proyecto se encuentra disponible de manera pública en la siguiente dirección web: <https://github.com/eon0111/RISC-V-CPU>.

4 El procesador monociclo

4.1. Descripción de la arquitectura de partida

El código que describe la composición y el funcionamiento interno del núcleo original se encuentra distribuido en cinco módulos funcionales, cada uno de los cuales implementa una parte diferenciada del camino de datos del procesador. Esta organización modular facilitará posteriormente la transformación del diseño en una arquitectura segmentada. Además, existe un módulo superior dentro del cual se establece el conexionado entre las diferentes etapas y, por encima de este, un módulo en un nivel adicional en el que se definen las conexiones entre el agregado de las unidades funcionales del pipeline y la memoria, unificada, la cual alberga datos e instrucciones.

Se proporciona a continuación, en la Figura 4.1, un diagrama con el que se espera poder ayudar a comprender las relaciones entre los distintos componentes que conforman el computador.

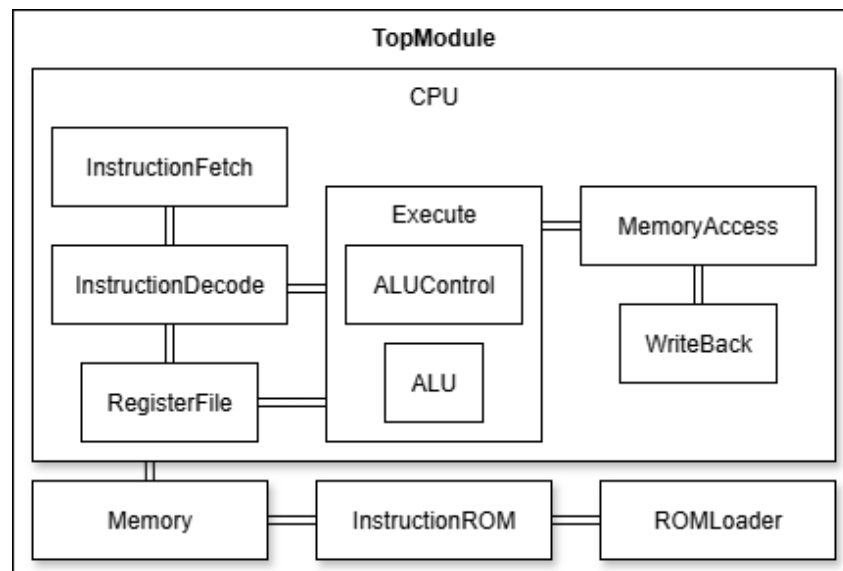


Figura 4.1: Diagrama de bloques de la arquitectura global del computador

A continuación se indicarán los módulos que conforman el diseño de partida junto con una descripción tanto de la funcionalidad que modelan, como de las señales de entrada y salida que reciben y generan respectivamente.

4.1.1. TopModule

TopModule es el módulo en el que se describe el conexionado de los grandes bloques que conforman el computador, es decir, el núcleo, la memoria, el cargador y la ROM de instrucciones, cuyo funcionamiento interno e interoperabilidad serán descritos en los subapartados siguientes a este.

El módulo cuenta con una interfaz de entrada-salida cuyos puertos permiten observar el estado del banco de registros y de la memoria en cualquier momento, lo cual resulta de gran utilidad cuando se necesita depurar la ejecución de programas complejos que puedan sacar a la luz defectos en el diseño del pipeline, los cuales no hubieran podido ser detectados por medio de los tests convencionales que prueban el comportamiento individual de cada uno de los componentes que conforman el diseño global.

4.1.2. Memory

Tal y como se indicaba al comienzo de este capítulo sobre la arquitectura interna del computador, la memoria se encuentra unificada, albergando tanto instrucciones como datos, lo cual contribuye a simplificar el diseño global, haciendo posible que todas las operaciones sobre la memoria sean realizadas por medio de una misma interfaz, definida dentro de un único módulo (*Memory*) que cuenta, además, con una instancia definida dentro del módulo *TopModule*.

Sobre el código del módulo *Memory*, en él se define la clase *Memory* con la que se construyen tanto la memoria unificada del computador, como la interfaz a la que se conecta el módulo *CPU* para habilitar la comunicación entre la memoria y los módulos que componen el núcleo, más concretamente, *InstructionFetch* y *MemoryAccess*, que emplean los puertos definidos en la interfaz de la memoria para leer de manera independiente, y en paralelo, los datos e instrucciones que en ella se encuentran alojados.

A continuación se detallará el propósito de los puertos de entrada-salida del módulo. No obstante, dentro de la entrada-salida se define también un *bundle* como una instancia de la clase *RAMBundle*, definida dentro del mismo fichero donde se especifica la clase *Memory*. Al instanciar esta clase se crea un *Bundle* de Chisel, es decir, una agrupación de señales de entrada-salida, en este caso, relacionadas con la interacción con la memoria (`address`, `write_data`, `write_enable` y `write_strobe`).

Señales de entrada

- ***instruction_address***: es un bus de 32 bits por donde la memoria recibe la dirección de la siguiente instrucción generada en el *fetch*.
- ***debug_read_address***: se trata de otro bus de 32 bits por el que la memoria puede recibir direcciones arbitrarias en cualquier instante de tiempo con fines de depuración.
- ***address@bundle***: la utilidad de este puerto, definido dentro del *bundle*, es indicar una dirección de memoria, cualquiera, para leerla o escribir en ella
- ***write_data@bundle***: es un bus de 32 bits mediante el cual se indica un dato a escribir en memoria
- ***write_enable@bundle***: una señal con la que habilitar o deshabilitar la escritura en memoria
- ***write_strobe@bundle***: tal y como puede verse en la Figura 4.2, donde se muestra el código con el que se implementa la clase *RAMBundle*, el tipo de dato del puerto de entrada *write_strobe* es un vector. Cada elemento de este vector es un booleano que indica si la sección n -ésima de la palabra a escribir en memoria, donde $0 \leq n < \text{Parameters.DataWidth}$, con $\text{Parameters.DataWidth} = 4$, deberá o no serlo. Por ejemplo, en las instrucciones de escritura de bytes individuales, el vector con el que se define el 'strobe' deberá tener la siguiente forma para indicar que deberá escribirse el último de los 4 bytes que conforman el dato: `{0001}`.

Señales de salida

- ***instruction***: la instrucción leída en la dirección *instruction_address*.
- ***debug_read_data***: la palabra leída en la dirección *debug_read_address*.
- ***read_data@bundle***: la palabra leída en la dirección *address@bundle*.

```

1 class RAMBundle extends Bundle {
2   val address      = Input(UInt(Parameters.AddrWidth))
3   val write_data   = Input(UInt(Parameters.DataWidth))
4   val write_enable = Input(Bool())
5   val write_strobe = Input(Vec(Parameters.WordSize, Bool()))
6   val read_data    = Output(UInt(Parameters.DataWidth))
7 }

```

Figura 4.2: Implementación de la clase RAMBundle

Sobre las memorias que proporciona Chisel, estas son direccionables a palabra, no a byte. Esta decisión puede deberse a la necesidad de dotar a la librería con la capacidad de generar memorias de cualquier tipología y tipo de direccionamiento, en función del tipo de dato con que se inicialice la memoria en el constructor, y en función de la máscara que se emplee en los accesos a estas memorias. Por ejemplo, una memoria inicializada del siguiente modo: `SyncReadMem(4096, Vec(4, 32))`, daría como resultado una memoria con 4096 entradas donde, cada entrada, albergaría 4 palabras de 32 bits cada una. Puede intuirse que este mecanismo de generación de memorias y acceso a las mismas podría posibilitar la construcción de caches, indicando el tag en el parámetro de dirección de los métodos de lectura/escritura.

4.1.3. InstructionROM y ROMLoader

Puede verse en el diagrama de la Figura 4.1 que el computador lo componen, además del núcleo y la memoria, dos piezas adicionales: *InstructionROM* y *ROMLoader*. Estos son dos módulos involucrados en la carga de binarios en memoria. El primero define internamente una rutina con la que se traslada el contenido del fichero binario resultante de un proceso de compilación cruzada, a un fichero de texto codificado en ASCII con el que puede inicializarse una memoria de Chisel, definida también a nivel interno dentro del módulo. Posteriormente, la lógica que define el comportamiento del módulo *ROMLoader* se encarga de trasladar, palabra a palabra, los contenidos de la ROM de instrucciones a la memoria de la CPU, a partir de un punto de entrada recibido como parámetro.

Por último, el módulo *ROMLoader* cuenta con una señal de salida con la que indica al módulo superior *TopModule* que la carga del programa en memoria ha finalizado y su ejecución podría dar comienzo. Mientras la carga del programa sigue en curso, el *bundle* de la memoria (*Memory*) permanece conectado a un bundle definido dentro de *ROMLoader*, de modo que pueda hacerse el volcado del contenido de la memoria definida dentro de *InstructionROM*, a la memoria que empleará más adelante el core. Cuando el cargador finaliza su trabajo, se desacopla de la memoria y esta se conecta al core ya con el programa cargado y listo para ejecutar.

En la Figura 4.3 se proporciona el fragmento del código de la clase *TOPModule* con el que se construye la lógica que define el comportamiento descrito en este último párrafo. Puede verse que el acoplamiento entre los *bundles* del cargador y la memoria, o entre esta y el núcleo, se realiza por medio del operador '`<>`', con el que se realiza el conexionado de señales con el mismo nombre.

```

1 when(!rom_loader.io.load_finished) {
2   rom_loader.io.bundle <> mem.io.bundle
3   cpu.io.memory_bundle.read_data := 0.U
4 }.otherwise {
5   rom_loader.io.bundle.read_data := 0.U
6   cpu.io.memory_bundle <> mem.io.bundle
7 }

```

Figura 4.3: Acoplamiento de la memoria en función del estado de la carga del programa.

4.1.4. CPU

Este módulo constituye uno de los grandes bloques que conforma el computador, el núcleo, y en su interior se describe el conexionado entre los distintos bloques que modelan cada una de las etapas por las que pasa una instrucción cuando es leída de la memoria. El funcionamiento de estas etapas se describirá en los subapartados siguientes a este.

Además de este conexionado, se define una interfaz de entrada-salida con los siguientes puertos:

Puertos de entrada

- ***instruction***: la instrucción leída de la memoria, en la dirección que genera el módulo *Instruction-Fetch*.
- ***instruction.valid***: una señal que controla el flujo de ejecución de nuevas instrucciones, en tanto que puede detener su lectura, paralizando el pipeline. Esta entrada se conecta al puerto de salida *load_finished* del cargador, con el que se indica que la carga de instrucciones en la memoria principal del computador ha finalizado, pudiendo dar comienzo a la ejecución del programa.
- ***debug_read_address***: un bus de 5 bits por el cual puede indicarse el índice de un registro cuyo valor quiera conocerse. Esta dirección es recibida por medio de su correspondiente puerto de depuración en el módulo superior (*TopModule*).

Puertos de salida

- ***debug_read_data***: el puerto de depuración donde el núcleo deposita el contenido del registro solicitado.
- ***instruction.address***: la dirección de la siguiente instrucción generada en el *fetch*.
- ***device.select***: el computador puede tener conectados hasta 8 dispositivos mapeados en memoria, cada uno de los cuales se identifica por medio de una etiqueta de 3 bits ($\log_2 8 = 3\text{bits}$), y es este el puerto por medio del cual se indica el dispositivo con cuyo espacio de memoria se quiere interactuar. En la práctica este puerto carece de utilidad, puesto que el único dispositivo con cuya memoria puede trabajarse desde el núcleo es la propia memoria principal. No obstante, queda claro que este mecanismo resultaría muy conveniente si se quisiera dotar al computador de comunicación de entrada-salida con una mayor variedad de dispositivos.

4.1.5. InstructionFetch

En este módulo se describe el comportamiento de la fase de *fetch*, donde se avanza el contador de programa a la siguiente instrucción sumándole 4 unidades¹, siempre y cuando se de alguna de las dos circunstancias siguientes:

- Que el cargador haya terminado de volcar el contenido de la ROM de instrucciones en la memoria principal de la CPU.
- Que el decodificador no haya determinado en el ciclo anterior que la instrucción en ese ciclo era un salto que debía tomarse.

En el primer caso el contador de programa permanecerá inmutable hasta el momento en que el cargador comunique la finalización del proceso de volcado y, en el segundo caso, el contador de programa tomará el valor del destino del salto.

¹Recordemos que se trata de un sistema 32 bits direccionable a byte, por lo que la siguiente instrucción se encontrará alejada 32 bits (4 bytes) respecto de la actual

Señales de entrada

- ***jump_flag_id***: es un bit que indica si el valor del contador de programa deberá ser igual al valor actual más cuatro unidades o, por el contrario, la dirección de destino de un salto que debe ser tomado.
- ***jump_address_id***: una señal de 32 bits por donde se recibe la dirección de destino calculada en la ejecución de las instrucciones de salto.
- ***instruction_read_data***: la instrucción leída de la memoria en la posición del contador de memoria.
- ***instruction_valid***: la señal con la que el cargador indica que su trabajo ha finalizado.

Señales de salida

- ***instruction_address***: la dirección de la instrucción a leer de la memoria en el siguiente ciclo.
- ***instruction***: la instrucción leída de la memoria en el ciclo actual. Se hace un *passthrough*, es decir, una asignación directa, entre este puerto y el puerto *instruction_read_data*.

Originalmente el código de este módulo se encontraba incompleto, puesto que carecía de la implementación de la lógica que establece el siguiente valor del contador de programa, es decir, el destino de un salto o la dirección de la siguiente instrucción en el programa. Esta lógica, mostrada en el código de la Figura 4.4, consiste en un multiplexor cuyo selector se conecta a la señal *jump_flag_id*, y cuyas dos entradas son, por un lado, el destino de un salto recibido por el puerto *jump_address_id* y, por otro lado, el valor del contador de programa más cuatro unidades (recordemos que se trata de una arquitectura de 32 bits).

```
1 pc := Mux(io.jump_flag_id, io.jump_address_id, pc + 4.U)
```

Figura 4.4: Conexión del contador de programa a la salida del multiplexor que determina su valor a cada ciclo en función de los saltos que se producen en el flujo de ejecución de los programas cargados en el computador

4.1.6. InstructionDecode

Como su nombre indica, esta es la clase que modela la lógica encargada de realizar la decodificación de las instrucciones recibidas desde el módulo de *fetch* para elaborar el conjunto de señales que componen la palabra de control, por medio de las cuales se gobernará el comportamiento del resto de unidades funcionales en el pipeline, de modo que estas actúen conforme al significado de las instrucciones decodificadas.

Se trata quizás del módulo con mayor complejidad de entre todos de los que se compone el pipeline, y esto se debe al elevado número de casuísticas que deben contemplarse en la generación de la palabra de control, que son el resultado directo de la variabilidad en cuanto a formatos de instrucciones dentro del ISA, así como de la variabilidad en el conjunto de valores posibles que pueden tomar los campos definidos en estos formatos.

Señales de entrada

- ***instruction***: la instrucción leída en la fase de *fetch*.

Señales de salida

- ***regs_reg{1,2}_read_address***: las direcciones de los registros cuyos valores se emplearán en el cómputo de los resultados de instrucciones aritmético-lógicas, o en la determinación del cumplimiento de la condición de salto en las instrucciones de salto condicional.
- ***ex_immediate***: inmediato con signo, extendido a 32 bits.
- ***ex_aluop{1,2}_source***: señales de control mediante las cuales se selecciona la procedencia de los operandos de la ALU.
- ***memory_{read,write}_enable***: señales de control mediante las cuales se habilita la lectura o escritura en la memoria.
- ***wb_reg_write_source***: señal de control con la que se selecciona el origen del dato a escribir en el banco de registros en la fase de *writeback* (resultado de la ALU, dato leído de memoria o siguiente contador de programa).
- ***reg_write_enable***: señal de control mediante la cual se habilita la escritura en el banco de registros.
- ***reg_write_address***: la dirección del registro en el que debe escribirse un resultado en la fase de *writeback*.

Dentro de este módulo tan solo fue preciso implementar la lógica correspondiente a la habilitación de la lectura o escritura en memoria, así como de la selección del origen del resultado a escribir en la fase de *writeback*, tal y como puede verse en el código expuesto en la Figura 4.5. Esta lógica se materializa por medio de un par de multiplexores: el primero de ellos gobierna la habilitación de la lectura en memoria y, el segundo, la escritura en memoria.

```
1 io.memory_read_enable := Mux(  
2   opcode == InstructionTypes.L,  
3   true.B,  
4   false.B  
5 )  
6 io.memory_write_enable := Mux(  
7   opcode == InstructionTypes.S,  
8   true.B,  
9   false.B  
10 )
```

Figura 4.5: Generación de las señales de habilitación de lectura/escritura en memoria

4.1.7. InstructionExecute

La clase *Execute* modela el comportamiento del hardware implicado en las operaciones que se llevan a cabo en la fase de ejecución y que son, por un lado, el cómputo de resultados de operaciones aritmético-lógicas en la ALU² y, por otro lado, la evaluación de las condiciones de salto junto con el cómputo del siguiente contador de programa en caso de resultar positivas esas evaluaciones.

La ALU se construye dentro del módulo *Execute* por medio de una instancia de la clase *ALU*, la cual cuenta con tres puertos de entrada (operación y primer y segundo operandos), y uno de salida (resultado). Además, su comportamiento es gobernado en base a las salidas generadas por una instancia de la clase *ALUControl*, por medio de cuya lógica se infiere el tipo de operación que deberá llevarse a cabo, en función de los valores que tomen los campos *opcode*, *funct3* y *funct7* en la instrucción.

A continuación se detallan las señales que conforman la entrada-salida del módulo, así como la función que desempeñan dentro de este.

²Arithmetic-Logic Unit: Unidad Aritmético-Lógica

Señales de entrada

- ***instruction***: la instrucción completa, de la cual se extraerán los campos *opcode*, *funct3* y *funct7*.
- ***instruction_address***: la dirección de la instrucción en ejecución. Este valor se empleará en el cómputo de direcciones efectivas de salto, así como del valor resultante de la ejecución de la instrucción *auipc*.
- ***reg1_data***: el valor leído en la dirección del primer registro fuente.
- ***reg2_data***: el valor leído en la dirección del segundo registro fuente.
- ***immediate***: el inmediato codificado en la instrucción, previamente extendido a 32 bits en la fase de decodificación.
- ***aluop1_source***: una señal de un bit con la que se indica cuál deberá ser el valor empleado como primer operando en la ALU. Esta señal controla la salida de un multiplexor cuyas entradas son la dirección de la instrucción y el valor leído en la dirección del primer registro fuente.
- ***aluop2_source***: una señal de un bit con la que se indica cuál deberá ser el valor empleado como segundo operando en la ALU. Esta señal controla la salida de un multiplexor cuyas entradas son el inmediato y el valor leído en la dirección del segundo registro fuente.

Señales de salida

- ***mem_alu_result***: el valor calculado en la ALU.
- ***if_jump_flag***: una señal de un bit con la que se indica el resultado de la evaluación de la condición de salto.
- ***if_jump_address***: en caso de resultar positiva la evaluación de la condición de salto, la dirección efectiva de este.

4.1.8. Memory Access

El módulo *MemoryAccess* incorpora la lógica por medio de la cual se gestionan los accesos a la memoria de datos en todas las variantes admitidas como parte de la especificación base, es decir, lecturas/escrituras de palabras completas, mitades o bytes individuales, con o sin extensión de signo.

A continuación se indicarán los puertos que componen la entrada-salida del módulo, así como su propósito y el uso que se hace de los datos que albergan. No obstante cabe mencionar que, como parte de la entrada-salida del módulo, se crea una instancia de la clase *RAMBundle* dentro de la cual se definen los puertos con los que poder interactuar con la memoria, tal y como se indicó en el apartado 4.1.2.

Señales de entrada

- ***alu_result***: la dirección en la cual se va a efectuar el acceso. Este mismo valor se pasa como entrada al bundle en su puerto *address*.
- ***reg2_data***: cuando va a realizarse una escritura, el dato a escribir.
- ***memory_read_enable***: un bit que habilita/deshabilita la lectura de datos en memoria.
- ***memory_write_enable***: un bit que habilita/deshabilita la escritura de datos en la memoria.
- ***funct3***: indica la longitud del dato con la que operar (palabra completa, mitad o byte).

Señales de salida

- ***wb_memory_read_data***: cuando se realiza una operación de lectura, el dato leído.

La memoria es direccionable a byte y, a partir de los dos³ últimos bits de la dirección puede computarse la posición del byte al cual se desea acceder. Ese valor, almacenado en la señal interna *mem_address_index*, se emplea del modo siguiente:

- Cuando se trata de **lecturas** de bytes individuales, con o sin extensión de signo, el índice se emplea para extraer la sección de 8 bits correspondiente de la palabra leída por medio del bundle. En el caso de las lecturas de media palabra, cuando el valor del índice es igual a 0 se lee la mitad menos significativa y, en cualquier otro caso, la más significativa.
- En las operaciones de **escritura** el índice se emplea en la construcción del *strobe* o máscara indicado en el bundle por medio del puerto *write_strobe* (ver Sección 4.1.2).

4.1.9. WriteBack

El módulo *WriteBack* consta de un multiplexor por medio del cual se selecciona el origen del dato a escribir en el banco de registros, bien sea un valor computado en la ALU, un dato leído de la memoria o el siguiente valor del contador de programa.

Señales de entrada

- ***alu_result***: el valor computado en la ALU.
- ***memory_read_data***: el dato leído de la memoria.
- ***instruction_address***: la dirección de la instrucción en curso.
- ***regs_write_source***: la señal de control que determina la salida en la fase de writeback.

Señales de salida

- ***regs_write_data***: el valor a escribir en el banco de registros.

${}^3\log_2 \left(\frac{32 \text{ bits (longitud de palabra)}}{8 \text{ bits (1 byte)}} \right) = 2$

5 El procesador segmentado

La segmentación (*pipelining*) es una técnica por medio de la cual la arquitectura de un computador se divide en múltiples etapas cada una de las cuales, se corresponde con una de las fases experimentadas por una instrucción desde que es leída de memoria hasta que sus resultados son escritos en el banco de registros. Este enfoque permite que distintas instrucciones ocupen distintas etapas de manera concurrente, explotando así el paralelismo a nivel de instrucción. Para lograrlo, deben introducirse en la arquitectura registros de etapa con los que almacenar de manera temporal el contexto de una instrucción al final de cada etapa, es decir, el conjunto de datos y señales de control asociados a la instrucción, los cuales serán transferidos a la etapa inmediatamente posterior en el siguiente ciclo de reloj.

Puede verse en la Figura 5.1 un diagrama en el que se compara el flujo de ejecución de un computador monociclo frente al de uno segmentado. Se trata de un caso ideal en el que el camino crítico de cada una de las cinco etapas en que se ha segmentado la arquitectura toma exactamente el mismo tiempo, y este es 1/5 del periodo de reloj del computador monociclo.

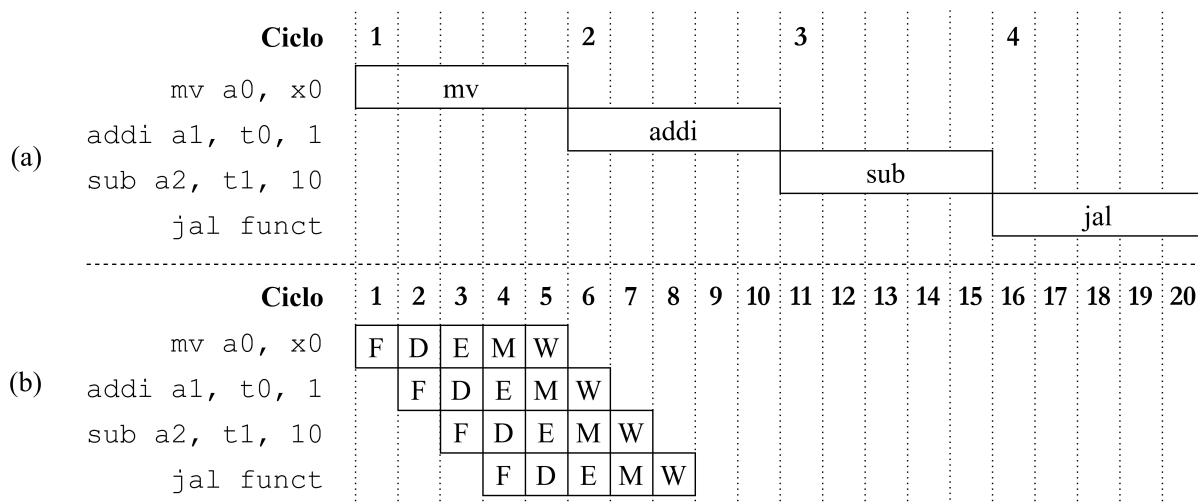


Figura 5.1: Ejecución de un programa en un computador con diseño monociclo (a) y segmentado (b)

Esta técnica conlleva una mayor complejidad arquitectónica y el surgimiento de una serie de riesgos o *hazards* generados bien por las dependencias de datos entre las instrucciones de un programa (ver Figuras 5.2 y 5.3), o bien por la ejecución de instrucciones de control que pueden o no alterar el flujo de ejecución de este (ver Figura 5.4). En función de las decisiones de diseño que se tomen para abordar la problemática que representan estos fenómenos, su aparición desembocará en la introducción de detenciones (*stalls*) en el flujo de ejecución, o la necesidad de incluir en el diseño mecanismos adicionales como el *forwarding*, consistente en el conexionado de determinadas señales de datos o de control hacia etapas anteriores en el pipeline.

Dependencias de datos

En la Figura 5.2 aparecen identificados los distintos tipos de dependencias de datos que pueden darse durante la ejecución de un programa. La dependencia RAW (a) se da cuando uno de los operandos de una instrucción es un registro (**x1**, señalado en color rojo) cuyo valor deberá ser previamente modificado por otra instrucción separada de la primera 2 ciclos o menos (ver Figura 5.3). Esto es así ya que la primera instrucción, la que lee el dato, deberá poder observar ese dato actualizado de modo que pueda elaborar un resultado consistente con el orden del programa. Es decir que, en un principio, la segunda instrucción no debería poder leer sus operandos (en su *decode*) hasta la escritura (*writeback*) del resultado calculado por la primera. Se ejemplifica esta situación por medio del diagrama presentado en la Figura 5.3.

add x1 , x2, x3 sub x4, x1 , x5 (a)	add x1 , x2, x3 add x1 , x4, x5 (b)	add x4, x1 , x2 add x1 , x3, x5 (c)
---	---	---

Figura 5.2: Ejemplos de riesgos de datos. (a) RAW (*Read After Write*): lectura tras escritura. (b) WAW (*Write After Write*): escritura tras escritura. (c) WAR (*Write After Read*): escritura tras lectura

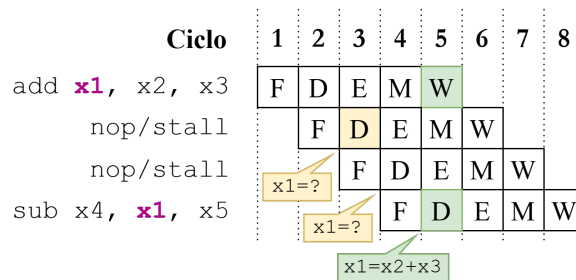


Figura 5.3: Flujo de ejecución de dos instrucciones implicadas en una dependencia RAW. Se señala en color verde el instante a partir del cual la segunda instrucción podrá leer el dato actualizado en el registro **x1** (ciclo 5).

Los otros dos tipos de dependencias presentados en la Figura 5.2, señalados con las letras *b* y *c*, no representan un riesgo real en arquitecturas en-orden como la utilizada en este trabajo. Esto es así ya que dos instrucciones pueden leer o escribir de manera consecutiva sobre el mismo registro sin que ello suponga la pérdida de coherencia en el dato, al igual que sucede con las escrituras sobre un registro que fue previamente leído.

Dependencias de control

En la Figura 5.4 se presenta un ejemplo de riesgo de control. En este ejemplo, la instrucción de resta **subi** entra en el pipeline antes incluso de que se haya evaluado la condición en la instrucción de salto **beq**, logrando completar su ejecución y dejando el valor del registro **t0** en un estado inconsistente. Además, dado que el destino del salto de la instrucción **loop** no se calculará hasta su ejecución, habrá dos ciclos en los que se introducirán las dos instrucciones inmediatamente posteriores (señaladas en color morado), una de las cuales reinicia el valor de la variable **a** (**t0**) haciendo que el bucle no tenga final. El resultado de la instrucción **subi** en la primera iteración se sobrescribe con el del segundo **addi**.

A pesar de las limitaciones comentadas, se debe destacar que la segmentación de instrucciones permite obtener un speedup teórico próximo al número de etapas en que se haya decidido segmentar la arquitec-

tura, teniendo siempre en cuenta que aspectos como la presencia y resolución de los riesgos mencionados, la latencia en los accesos a memoria o el tiempo desigual entre los caminos críticos de las diferentes etapas, impedirán alcanzar ese rendimiento máximo ideal.

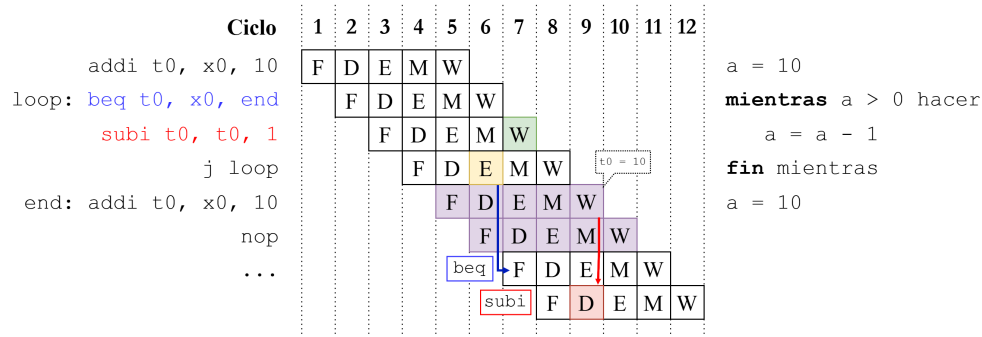


Figura 5.4: Representación del flujo de ejecución de un programa sobre una arquitectura segmentada sin mecanismos para resolver riesgos de control.

5.1. Arquitectura de un registro de etapa

La función de un registro de etapa es almacenar de manera transitoria el conjunto de señales de control y datos emitidos al término de una etapa de modo que, en el siguiente ciclo de reloj, esas mismas señales puedan ser transferidas a la etapa inmediatamente posterior en el pipeline. De este modo se consigue desacoplar temporalmente la ejecución de las distintas etapas permitiendo que varias instrucciones ocupen el pipeline de manera concurrente.

Desde el punto de vista de la implementación en Chisel, un registro de etapa como el mostrado en la Figura 5.5 se compone, en primer lugar, de un bundle en el que se definen las entradas y salidas de este. Estas se corresponderán con el conjunto de señales de control y datos que deban avanzar de una etapa a la siguiente. Posteriormente, cada entrada de datos/control se conectará a la entrada de un registro específico de esa señal y, la salida del registro, a su correspondiente salida en el bundle.

```

1 class FD extends Module {
2
3   val io = IO(new Bundle {
4     val f_instruction      = Input(UInt(Parameters.InstructionWidth))
5     val f_instruction_address = Input(UInt(Parameters.AddrWidth))
6     val d_instruction      = Output(UInt(Parameters.InstructionWidth))
7     val d_instruction_address = Output(UInt(Parameters.AddrWidth))
8   })
9
10  val instruction      = RegInit(0.U(Parameters.InstructionWidth))
11  val instruction_address = RegInit(0.U(Parameters.AddrWidth))
12
13  instruction      := io.f_instruction
14  instruction_address := io.f_instruction_address
15  io.d_instruction      := instruction
16  io.d_instruction_address := instruction_address
17 }

```

Figura 5.5: Implementación inicial del registro de etapa que separa las fases de *fetch* y decodificación.

5.2. Segmentando el núcleo en dos etapas

El primer paso hacia la segmentación del núcleo en las cinco etapas marcadas como objetivo del trabajo consistió en segmentarlo primeramente en dos, aislando la fase de *fetch* respecto de las unidades funcionales implicadas en el resto de etapas.

Esta división no conlleva la aparición de riesgos de datos puesto que tanto la escritura como la lectura de estos forman parte de la misma fase dentro del ciclo de procesamiento de las instrucciones. Sin embargo, esta segmentación sí da pie a que, durante la ejecución de un programa, puedan aparecer riesgos de control tal y como se muestra en el ejemplo presentado en la Figura 5.6, donde la ausencia de mecanismos que logren mitigar el riesgo de control provoca la lectura de la instrucción señalada en color rojo antes de que se haya evaluado siquiera la condición en la instrucción de salto, ejecutando así la instrucción de resta y depositando en el registro `x1` un valor incoherente con la lógica del programa.

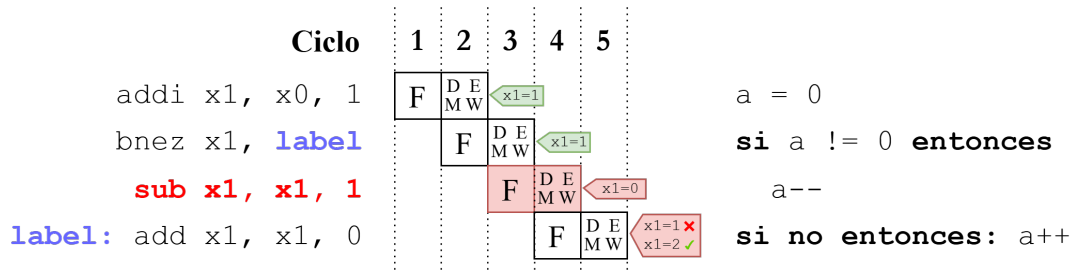


Figura 5.6: Representación del flujo de ejecución de un programa sobre una arquitectura segmentada en dos etapas sin mecanismos para resolver riesgos de control.

Para verificar la existencia o no del riesgo de control tras la introducción en la arquitectura del primer registro de segmentación, se desarrolló un sencillo programa en lenguaje ensamblador el cual incorpora instrucciones de salto condicional, salto incondicional y una llamada a función. Este programa (cuyo código se muestra en la Figura 5.7) fue compilado para RISC-V, cargado en memoria y ejecutado en el computador simulado por medio del test de Chisel mostrado en la Figura 5.8. El test consta, en primer lugar, de un bucle de inicialización en el que se espera a que el cargador lea el contenido del binario y lo deposite en la memoria del computador. Posteriormente, se ejecuta un segundo bucle en el que se hace avanzar el reloj 100 ciclos, a priori, suficientes como para recoger la ejecución completa del programa en la traza VCD resultante de la ejecución del test.

Código máquina	Ensamblador	Pseudocódigo
<pre> 00000000 <_start>: 0: 00000313 4: 00400393 00000008 <loop_start>: 8: 0063ca63 c: 00030513 10: 010000ef 14: 00050313 18: ff1ff06f 0000001c <loop_end>: 1c: 0000006f 00000020 <suma_uno>: 20: 00050293 24: 00128293 28: 00028513 2c: 00008067 </pre>	<pre> li t1,0 li t2,4 blt t2,t1,loop_end mv a0,t1 jal suma_uno mv t1,a0 j loop_start j loop_end mv t0,a0 addi t0,t0,1 mv a0,t0 ret </pre>	<pre> t1 = 0 t2 = 4 mientras t1 < t2 t1 = suma_uno(t1) función suma_uno(x) retornar x + 1 </pre>

Figura 5.7: Desensamblado, código ensamblador y pseudocódigo de un programa sencillo.

```

1 class SegRegFDTest extends AnyFlatSpec with ChiselScalatestTester {
2   behavior.of("Pipelined CPU")
3   it should "run a simple program" in {
4     test(new TestTopModule("simple_subroutine.asmbin"))
5       .withAnnotations(TestAnnotations.annos) { c =>
6         while(!c.io.instruction_valid.peek().litToBoolean) {
7           c.clock.step()
8           c.io.mem_debug_read_address.poke(4.U)
9         }
10
11         for (i <- 1 to 100) {
12           c.clock.step()
13           c.io.mem_debug_read_address.poke((i * 4).U)
14         }
15
16         c.io.regs_debug_read_address.poke(6.U) // x6 => t1
17         c.io.regs_debug_read_data.expect(5.U)
18       }
19   }
20 }

```

Figura 5.8: Código del test con el que se pone a prueba el funcionamiento de un programa sencillo sobre el núcleo segmentado en dos fases.

El primero de los problemas que se observó al examinar la traza generada por el test fue la ‘falta de sincronismo’ entre el valor del contador de programa observado en una etapa determinada (el *pc* se propaga por el pipeline), y la instrucción observada en esa misma etapa. Esto es, que dada una instrucción observada en una fase determinada del pipeline, el valor del contador de programa que se observa en esa misma fase no se corresponde con el de la instrucción, si no con el de la instrucción que le precede en el programa.

En la Figura 5.9 se muestra una sección de la traza obtenida tras la ejecución del test de la Figura 5.8. En el punto de la ejecución que se señala en color rojo se observa un ejemplo de este comportamiento y, para mayor comprensión de la traza, se indica a continuación la función de las señales mostradas:

- **instr_addr@fetch**: la dirección de la instrucción presente en la fase de fetch.
- **instr_addr@DEMW**: la dirección de la instrucción en la fase siguiente al fetch.
- **instr@fetch**: la instrucción leída en la última fase de fetch.
- **instr@DEMW**: la instrucción leída en el anterior fetch.
- **fetch_io_jump_flag**: la señal que indica si la evaluación de una condición de salto es positiva. El valor de esta señal determina si en el siguiente ciclo se hace *fetch* de la instrucción en el siguiente contador de programa o del *target* de un salto. Tanto esta señal como el *target* se obtienen en la fase de ejecución.
- **ex_io_immediate**: el offset empleado en el cálculo de la dirección efectiva de salto.
- **ex_io_instr_addr**: la dirección base del salto.
- **fetch_io_jump_addr**: la dirección efectiva del salto. Esta señal llega al módulo *InstructionFetch* desde *Execute*.

Este comportamiento supone un gran inconveniente a la hora de realizar el cálculo de la dirección de destino de las instrucciones de salto. Cuando un salto pasa a la fase de ejecución, se calcula el destino tomando como operandos en la ALU una dirección base (la propia del salto) y un desplazamiento a partir de esa dirección. Si la dirección base no se corresponde con la del salto en fase de ejecución, el destino calculado será erróneo.

Tal y como se observa en la traza de la Figura 5.9, al comienzo del ciclo marcado en la posición en que se encuentra el indicador vertical (43 ps), la instrucción `jal suma_uno (0x010000EF @ 0x10101`, ver señal `instr@DEMW`), observa los siguientes operandos en el cálculo del destino del salto:

- `ex_io_immediate`: 0x10
- `instr_addr@DEMW`: 0x1014

Puede observarse que la dirección base del salto no es la correspondiente a la instrucción `jal`, si no la de la instrucción `mv` que la sucede en el programa, provocando que el salto no sea al comienzo de la subrutina `suma_uno`, si no a la segunda instrucción dentro de esta.

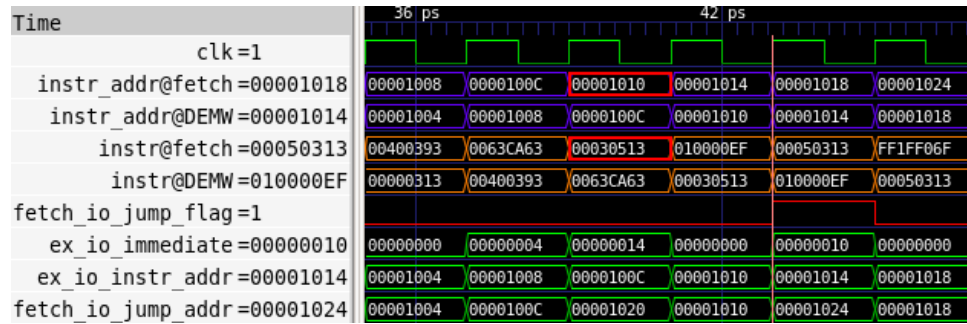


Figura 5.9: Sección de la traza obtenida tras la ejecución del test mostrado en la Figura 5.7

El diseño original emplea como memoria una instancia de la clase *SyncReadMem*. Este tipo de memoria no permite realizar lecturas de manera asíncrona, lo cual implica que, cuando se solicita una lectura, el dato leído no sale de la memoria hasta el siguiente flanco ascendente de reloj, impidiendo que se mantenga la sincronía entre las instrucciones en cada fase y la dirección de la instrucción observada en esa misma fase. En el computador objeto de este trabajo, este comportamiento de la memoria se materializa en que, cuando se envía la dirección de una instrucción para leerla, esa lectura no será efectiva hasta el ciclo siguiente al envío de la dirección. De ahí que, por ejemplo, la dirección de una instrucción en la fase de fetch (señal `instr_addr@fetch`) no se corresponda con la instrucción observada en esa misma fase (señal `instr@fetch`), si no con la instrucción inmediatamente anterior a la solicitada.

Para solucionar esta problemática se cambia, dentro de la clase *Memory*, el tipo de memoria a una de la clase *Mem*, de forma que la lectura del dato se lleve a cabo en el instante mismo en que se solicita. De este modo, tal y como puede observarse en la traza presentada en la Figura 5.10, el valor de la dirección de la instrucción en cada fase se corresponde con la de la instrucción que se está procesando en esa misma fase, de tal modo que ahora el cálculo del *target* en la instrucción de salto sea correcto ($0x1010$ (`ex_io_instr_addr`) + $0x10$ (`ex_io_immediate`) = $0x1020$ (`fetch_io_jump_addr`) → `suma_uno`).

A pesar de esta corrección, puede observarse que el flujo de ejecución del programa sigue siendo erróneo ya que, siguiendo a la instrucción de salto (tras la marca vertical), en la segunda fase logra introducirse y ejecutarse la instrucción inmediatamente posterior, la cual es un `mv t1, a0 (0x00050313 @ 0x1014)`. En el caso concreto de la primera iteración del bucle, la ejecución de esta instrucción es inócua ya que en ese punto los registros `t1` y `a0` albergan el mismo valor. No obstante, la inserción de la instrucción siguiente al salto toma una relevancia significativa cuando se trata de otra instrucción de salto, tal y como ocurre tras el salto incondicional al final del bucle. La ejecución de este segundo salto, el cual forma parte del bucle de final de programa, provoca que este llegue a término tras la primera iteración (véase Figura 5.9).

¹El punto de entrada de los programas se encuentra en la dirección `0x1000`. La dirección de la instrucción es el valor del punto de entrada sumado al offset mostrado en la primera columna de la Figura 5.7

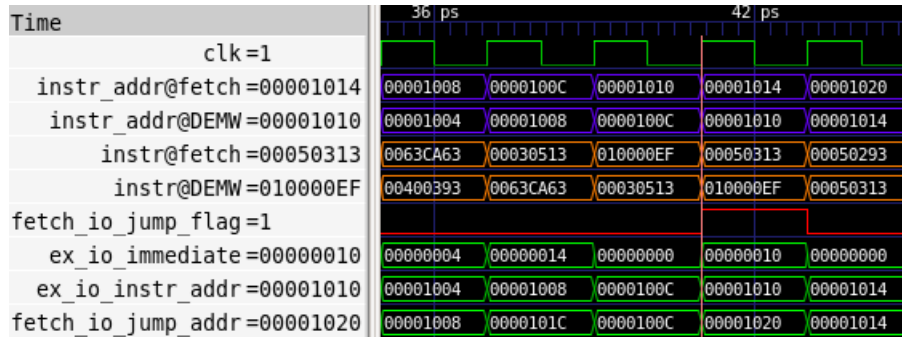


Figura 5.10: Misma sección de la traza pero con una instancia de la clase Mem como entidad de memoria en lugar de SyncReadMem.

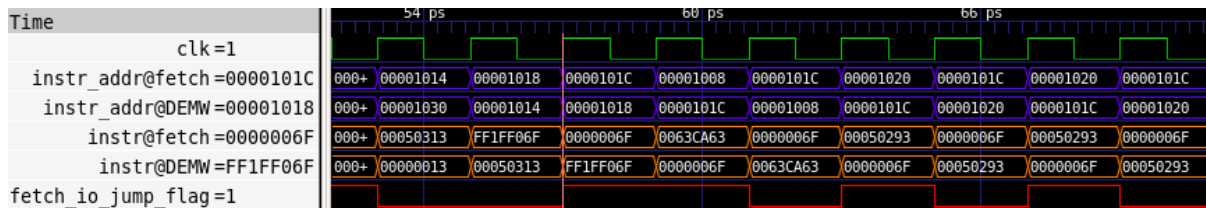


Figura 5.11: Sección de la traza en la que se muestra la pronta ejecución del bucle final.

La solución final para resolver esta problemática consiste en la introducción de un par de modificaciones en la fase de fetch para hacer que la arquitectura introduzca una operación `nop` (0x00000013) tras cada instrucción de salto en cuya segunda fase se determine que este deba ser tomado. Estas modificaciones se realizan sobre la lógica que hace avanzar el contador de programa.

En la nueva implementación, tal y como se observa en el `diff` presentado en la Figura 5.12, se introduce un `nop` en el pipeline siempre y cuando se cumplan las dos condiciones siguientes: en primer lugar, que el cargador no haya terminado su trabajo y, en segundo lugar, que en la segunda fase de una instrucción de salto se haya determinado que este deba ser tomado (originalmente solo se contemplaba la primera condición).

```

val pc = RegInit(ProgramCounter.EntryAddress)

- when(io.instruction_valid) {
-   pc          := Mux(io.jump_flag_id, io.jump_address_id, pc + 4.U)
-   io.instruction := io.instruction_read_data
+ when(io.instruction_valid && !io.jump_flag_id) {
+   pc          := pc + 4.U
+   io.instruction := io.instruction_read_data
+ } . otherwise {
-   pc          := pc
+   pc          := io.jump_address_id
+   io.instruction := 0x00000013.U
+ }

io.instruction_address := pc

```

Figura 5.12: Comparación entre la implementación original del módulo `InstructionFetch` y su adaptación para resolver riesgos de control.

5.3. Segmentando el resto del núcleo

En un primer momento se optó por aplicar un enfoque incremental en lo que respecta a la inserción de los registros de etapa. No obstante, este enfoque se aplicó tan solo en la integración del primer registro de etapa para separar la fase de *fetch* del resto del pipeline.

La integración de los registros de etapa uno a uno hubiera requerido verificar el funcionamiento del núcleo con dos, tres y cuatro registros, y el proceso de desarrollo se hubiera dilatado en el tiempo más allá de lo deseado inicialmente. Además, esta aproximación habría supuesto, en cada paso, la implementación de una solución distinta a los problemas derivados de las dependencias de datos y de control entre las instrucciones presentes en el pipeline. Por ello se consideró que resultaría más eficiente integrar los registros de segmentación siguientes al de *fetch-decode* a la vez, desarrollando una única solución.

Esta solución se ha materializado en el código del módulo *HazardUnit*, cuyo cometido es observar el estado de la arquitectura a cada ciclo para identificar y resolver las dependencias de datos o de control que pudieran originarse durante la ejecución de los programas. A continuación se detalla el método empleado en la resolución de los riesgos de datos del tipo RAW y de control, así como las señales de la *hazard unit* involucradas en el proceso de detección y mitigación del riesgo.

5.3.1. Resolución de riesgos de datos (RAW)

En el caso de una arquitectura segmentada en cinco fases, un riesgo de datos del tipo RAW se da cuando en un programa existe una pareja de instrucciones consecutivas para las cuales se cumple que, la segunda de ellas, posee como registro origen en alguno de sus operandos el registro de destino de la primera, pudiendo encontrarse separadas una distancia de 1 o 2 ciclos. Esta casuística queda ilustrada por medio del ejemplo presentado en la Figura 5.13, donde la instrucción `subi` debe esperar al *writeback* del `addi` para poder leer del registro `x10` un valor actualizado y consistente con la lógica del programa.

Ciclo	1	2	3	4	5	6	7	8
<code>addi x10, x10, 0x1</code>	F	D	E	M	W			
<code>subi x10, x10, 0x1</code>		F	D	E	M	W		
<code>subi x10, x10, 0x1</code>			F	D	E	M	W	
<code>subi x10, x10, 0x1</code>				F	D	E	M	W

Figura 5.13: Ejemplo de una dependencia RAW en una arquitectura segmentada en cinco fases.

La solución (que no sea introducir *nops*) al problema que supone la ocurrencia de esta clase de escenarios pasa por la aplicación de una técnica denominada *forwarding*. Esta técnica consiste en adelantar el resultado de una instrucción, ya sea aritmético-lógica o una lectura de memoria, directamente a la fase de ejecución de las instrucciones que deban emplear ese dato como operando. En el ejemplo de la Figura 5.13, la instrucción `subi` podría ejecutar sin problemas justo después del `addi` si se adelantase el resultado del `addi` desde la fase de memoria. El forwarding se haría desde la fase de memoria del `addi` a la fase de ejecución del `subi`, de tal modo que se pueda ejecutar la segunda instrucción inmediatamente después de la primera sin que sea necesario introducir *nops*.

A continuación, se indicarán las soluciones propuestas al problema de las dependencias de datos del tipo RAW, tanto en el caso de las instrucciones aritmético-lógicas como en el caso de la instrucción de lectura de memoria:

Riesgos causados por la sucesión de instrucciones aritmético-lógicas

Las señales de la *hazard unit* implicadas en la detección de esta clase de riesgos son:

- **rs1_ex**: la dirección del registro empleado como primer operando de la instrucción, observado en la fase de ejecución.

- **rs2_ex**: la dirección del registro empleado como segundo operando de la instrucción, observado en la fase de ejecución.
- **rd_mem**: la dirección del registro destino, observado en la fase de acceso a memoria.
- **rd_wb**: la dirección del registro destino, observado en la fase de escritura de resultados.
- **rd_wb**: la dirección del registro destino, observado en la fase de escritura de resultados.
- **rd_wb**: la dirección del registro destino, observado en la fase de escritura de resultados.

La *hazard unit* gobierna, además, las señales de selección de dos multiplexores emplazados, cada uno, en una de las entradas de la ALU:

- **alu_rs1_src**: controla la selección del primer operando de la ALU, que puede ser un dato leído a partir del banco de registros por parte de la instrucción en fase de ejecución, o un resultado adelantado desde la fase de memoria o de *writeback*.
- **alu_rs2_src**: exactamente igual que la anterior, pero aplicada al segundo operando de la ALU.

La lógica que rige el proceso de detección de las dependencias por parte de la *hazard unit* es tal y como se indica en la Figura 5.14, siendo el código aplicable tanto en el caso en que la dependencia se diese con el primer registro operando, como si esta se diese con el segundo.

Más allá de lo auto-explicativo del diagrama de flujo que acompaña al código, cabe reseñar el por qué de la condición en la que se evalúa si el registro origen es `x0` y es que, de darse una dependencia RAW en la que este registro se encontrase involucrado, no tendría sentido realizar un *forwarding* de un registro cuyo valor es constantemente 0, por lo que se evita solventar esta falsa dependencia.

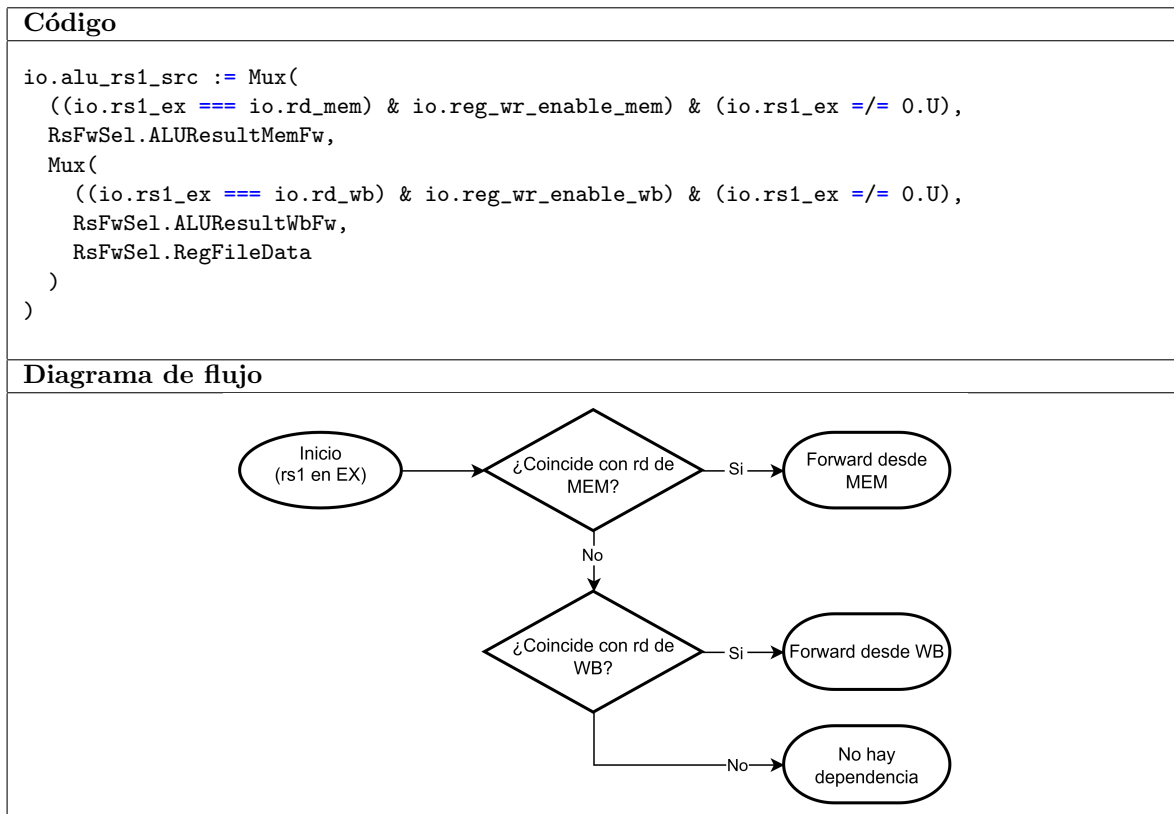


Figura 5.14: Implementación en Chisel y diagrama de decisión de la lógica de *forwarding* para la selección del primer operando de la ALU.

Dependencias en las que se encuentra involucrada la instrucción 'lw'

La presencia de la instrucción `lw` en una dependencia de datos del tipo RAW genera un caso especial que no puede resolverse por medio de la metodología seguida en el apartado anterior. Esto es debido a que el resultado de la instrucción `lw` no se genera hasta haber completado la lectura de un dato de memoria, lo cual se produce al término de la fase de memoria de la instrucción. En ese momento la instrucción dependiente inmediatamente posterior al `lw` ya habrá completado su fase de ejecución, pero lo habrá hecho con un operando erróneo.

La técnica con la que se resuelve la dependencia consiste en detener el procesamiento de las instrucciones en fases de *fetch* y decodificación cada vez que esta sea detectada. Esto se consigue reteniendo los datos del registro de segmentación que separa las fases de *fetch* y decodificación, y el contador de programa, y haciendo *flush* del registro de segmentación que separa decodificación y ejecución. Esta retención y limpieza dura tan solo un ciclo, de tal modo que, cuando la instrucción dependiente que no es el `lw` pase a la fase de ejecución, la *hazard unit* tratará la dependencia como si de cualquier otra dependencia de datos se tratase, adelantando desde el writeback del `lw` a la entrada de la ALU que corresponda, el valor leído de la memoria. Se proporciona en la Figura 5.15 una representación de la resolución del riesgo de datos por medio de esta técnica.

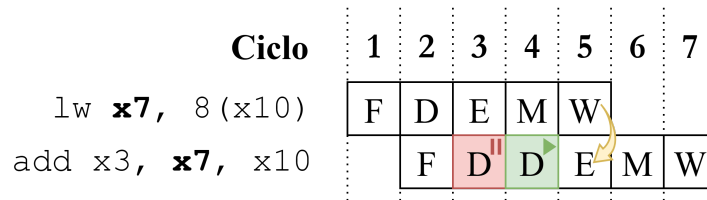


Figura 5.15: Flujo de ejecución de dos instrucciones implicadas en una dependencia de datos de tipo RAW, una de las cuales es la instrucción `lw`.

La detención y limpieza de los registros de segmentación mencionados, así como la parada del del contador de programa, se controlan por medio de la señal `lw_stall`, cuyo cómputo se realiza según lo indicado en la primera línea del código de la Figura 5.16. La expresión lógica que determina el valor de la señal se compone de las siguientes tres condiciones:

- $C_1 \rightarrow io.ex_regs.write_src(0)$: en primer lugar, se identifica si una instrucción en fase de ejecución va a provocar la escritura en el banco de registros de un dato leído de memoria. Esto se hace observando el bit menos significativo de la señal `ex_regs.write_src`, la cual controla la salida del multiplexor en la fase de *writeback* con el que se escoge el dato a escribir en el banco de registros. La salida de este multiplexor podrá ser un resultado calculado en la ALU, la dirección de retorno de un salto o el dato leído en una posición de memoria. En este último caso, el valor del bit menos significativo de la señal `ex_regs.write_src` es igual a 1.
- $C_2 \rightarrow io.rs1_d == io.rd_ex$: detección de dependencia de datos RAW entre dos instrucciones consecutivas emplazadas en las fases de decodificación y ejecución. Se observa la dirección del primer registro origen.
- $C_2 \rightarrow io.rs2_d == io.rd_ex$: exactamente igual que la condición anterior, pero observando la dirección del segundo registro origen.

En resumen, la expresión lógica con la que se define el valor de la señal `lw_stall` es como sigue:

$$lw_stall = C_1 \wedge (C_2 \vee C_3)$$

Si se detecta una dependencia de datos RAW entre instrucciones emplazadas en las fases de decodificación y ejecución (C_2 o C_3) y, además, se detecta un `lw` en la fase de ejecución (C_1), se detienen el contador de programa y el registro de segmentación *fetch*-decodificación, y se hace `flush` del registro de segmentación de decodificación-ejecución. De este modo, tan solo el `lw` podrá avanzar en el pipeline. En el ciclo siguiente a la detección de la dependencia, el `lw` leerá el dato de memoria, el cual podrá ser adelantado, en el ciclo posterior, a la fase de ejecución de la otra instrucción implicada en la dependencia.

```

1 val lw_stall = io.ex_regs_write_src(0) &
2               ((io.rs1_d === io.rd_ex) | (io.rs2_d === io.rd_ex))
3
4 io.pc_stall    := lw_stall
5 io.srFD_stall := lw_stall
6 io.srDE_flush := lw_stall

```

Figura 5.16: Cómputo de la señal `lw_stall`.

Las señales `pc_stall`, `srfD_stall` y `srDE_flush` controlan, respectivamente, la parada del contador de programa, la detención del registro de segmentación de *fetch*-decodificación, y la limpieza del registro de segmentación de decodificación-ejecución.

Al instanciar el registro de segmentación en el módulo CPU, se indica que la señal de *reset* del registro será la señal `srDE_flush` generada por la *hazard unit*, tal y como se observa en el código mostrado en la Figura 5.17.

```

1 val srDE = withReset(hazard_unit.io.srDE_flush){Module(new DE)}

```

Figura 5.17: Instanciación, en el módulo CPU, del registro de segmentación que separa las fases de decodificación y ejecución.

Por último, se ha adaptado la lógica del módulo `InstructionFetch` (ver Figura 5.18) de tal modo que el contador de programa avance tan solo cuando el cargador haya indicado que ha terminado su trabajo (valor 1 en el puerto `instruction_valid`), y la *hazard unit* lo permita (valor cero en el puerto `pc_stall`).

```

1 when(io.instruction_valid & !io.pc_stall) {
2   io.instruction := io.instruction_read_data
3   pc := Mux(io.jump_flag_id, io.jump_address_id, pc_plus_four)
4 } .elsewhen(io.instruction_valid & io.pc_stall) {
5   pc := pc
6   io.instruction := io.instruction_read_data
7 } .otherwise {
8   pc := pc
9   io.instruction := 0x00000013.U
10 }
11
12 io.current_pc := pc
13 io.next_pc    := pc_plus_four

```

Figura 5.18: Adaptación del código del módulo `InstructionFetch`.

5.3.2. Resolución de riesgos de control

Una solución sencilla y eficaz al problema generado por los riesgos de control es limpiar el contenido de los registros de segmentación entre las etapas de *fetch*-decodificación y decodificación-ejecución cada

vez que, en la fase de ejecución de una instrucción de salto, se determine que este deba ser tomado. De este modo, tal y como se observa en el ejemplo de la Figura 5.19, todas las instrucciones (máximo 2) que logren introducirse en el pipeline tras la instrucción de salto, no se procesarán completamente y su paso por la arquitectura habrá sido totalmente inocuo.

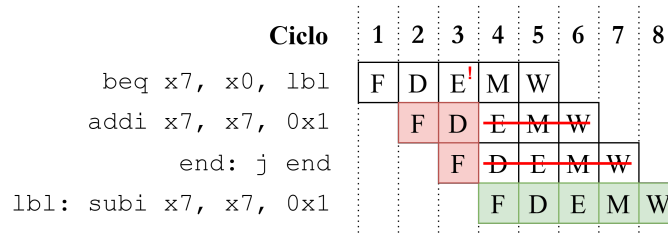


Figura 5.19: Diagrama ilustrativo del proceso de mitigación de los riesgos de control.

Esta solución se implementa por medio del fragmento de código de la *hazard unit* mostrado en la Figura 5.20, donde se establece que el valor de la señal de borrado de los registros de segmentación indicados (señales `srDE_flush` y `srFD_flush`²), será igual al valor del flag de salto observado en la fase de ejecución.

```

1 io.srDE_flush := lw_stall | io.ex_jump_flag
2 io.srFD_flush := io.ex_jump_flag

```

Figura 5.20: Configuración de la señal de borrado de los registros de segmentación que separan las fases de fetch-decodificación y decodificación-ejecución, conforme al valor del flag de salto en fase de ejecución.

²La instanciación del registro de segmentación de *fetch*-decodificación en el módulo CPU se lleva a cabo de igual modo que en el caso del registro `srDE`, indicando que el *reset* del registro será la señal `srFD_flush` generada en la *hazard unit*

6 Despliegue de las implementaciones sobre FPGA

La fase final del proyecto consiste en la adaptación de las implementaciones con las que se ha trabajado, de tal modo que estas pueden ser cargadas en el chip FPGA de la placa de desarrollo Arty A7 100T. Además, se pretende realizar una prueba de concepto en la que se envíe un código de ejemplo en formato binario por medio de la interfaz serie disponible en la placa. El código será posteriormente cargado en la memoria del computador para ser posteriormente ejecutado. Esto requerirá incluir una serie de modificaciones en el diseño de la CPU de tal modo que la tarea de realizar la inicialización de la memoria ya no recaiga en el cargador, si no en un nuevo módulo que dote al computador de capacidad para comunicar, usando el protocolo UART, con un host externo del cual recibirá el código de prueba. Esta comunicación se iniciará en el host remoto por medio de la utilidad *TeraTerm*.

Se muestra en la Figura 6.1 el diragrama de la arquitectura global del computador, actualizado para sustituir la memoria ROM de instrucciones y el cargador (*InstructionROM* y *ROMLoader* respectivamente), por un nuevo módulo *UART*.

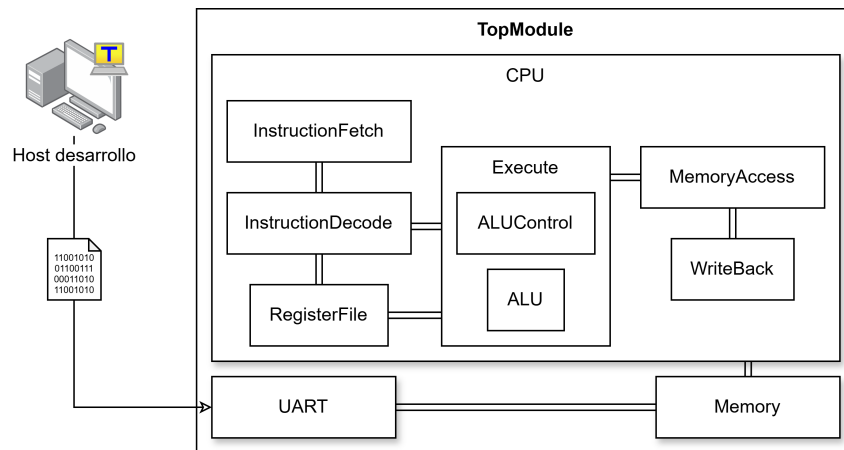


Figura 6.1: Diagrama de bloques con los módulos de los que se compone el computador.

6.1. Proceso de despliegue

6.1.1. Obtención del HDL de bajo nivel

Por defecto, un fichero de código desarrollado en Chisel no cuenta con un método principal que pueda ejecutarse. Esto es así puesto que la finalidad del código es solamente describir, por medio de constructos propios del lenguaje Scala, el comportamiento interno de componentes hardware. No obstante a esto,

el flujo de despliegue de un circuito digital sobre hardware real usando el software Vivado, exige que el punto de entrada de dicho flujo sea código Verilog/VHDL.

Para obtener código Verilog a partir de un desarrollo implementado en Chisel [27] es necesario añadir la declaración de objeto mostrada en las líneas 35 a 37 de la Figura 6.2. La instanciación de ese objeto *Top* extendiendo la clase *App* dota al código compilado de un punto de entrada en el que se ejecuta el método `emitVerilog`, cuyo resultado será una traducción, en sintaxis Verilog, del código Chisel con que se define el módulo *Top*.

A partir de este fichero Verilog, Vivado llevará a cabo las etapas de síntesis, implementación y generación del *bitstream*¹ que posteriormente se programará en el chip FPGA de la placa de desarrollo.

```

1 class Top extends Module {
2
3   val io = IO(new Bundle {
4     val instruction_valid = Input(Bool())
5     val i_uart_rx        = Input(Bool())
6     val pc_val           = Output(UInt(4.W))
7   })
8
9   val cpu  = withReset(reset.asAsyncReset) { Module(new CPU) }
10  val mem  = Module(new Memory(8192))
11  val uart = Module(new UART)
12
13  withReset(~reset.asBool) {
14    uart.io.sw_tx      := io.instruction_valid
15    uart.io.i_uart_rx := io.i_uart_rx
16
17    mem.io.debug_read_address := 0.U
18    cpu.io.instruction_valid  := io.instruction_valid
19    mem.io.instruction_address := cpu.io.instruction_address
20    cpu.io.instruction        := mem.io.instruction
21
22    cpu.io.debug_read_address := 7.U
23    io.pc_val := cpu.io.instruction_address
24
25    when(!io.instruction_valid) {
26      uart.io.mem_bundle <> mem.io.bundle
27      cpu.io.memory_bundle.read_data := 0.U
28    } .otherwise {
29      uart.io.mem_bundle.read_data := 0.U
30      cpu.io.memory_bundle <> mem.io.bundle
31    }
32  }
33 }
34
35 object Top extends App {
36   (new chisel3.stage.ChiselStage).emitVerilog(new Top())
37 }

```

Figura 6.2: Implementación de la CPU completa, con la definición del punto de entrada a la función que genera el circuito en lenguaje Verilog.

¹El bitstream describe cómo debe configurarse el tejido programable de la FPGA, incluyendo la configuración de LUTs, registros, bloques de memoria, redes de interconexión y recursos de entrada/salida.

El computador cuenta con las siguientes señales de entrada y salida, las cuales han sido añadidas para trabajar con la FPGA:

- ***instruction_valid* (entrada)**: La señal con la que se establece si el computador debe permanecer a la espera de recibir el programa o si, por el contrario, debe dar comienzo a su ejecución.
- ***i_uart_rx* (entrada)**: la entrada de datos del módulo UART.
- ***pc_val* (salida)**: el valor del contador de programa.

6.1.2. Configuración del fichero de *constraints*

Para asociar la entrada/salida del módulo al hardware disponible en la placa es necesario crear un fichero de *constraints* en el que se indique, por cada puerto de entrada/salida en el módulo, el componente hardware al cual se conectará físicamente.

Se muestran en la Figura 6.3 las asignaciones realizadas para probar el diseño final.

```

1  # Se mapea la señal de control de inicio del programa sobre uno de los interruptores deslizantes
2  set_property -dict { PACKAGE_PIN A8  IOSTANDARD LVCMOS33 } [get_ports { io_instruction_valid }];
3
4  # Se mapea la entrada de datos el módulo UART a la salida del UART de la placa
5  set_property -dict { PACKAGE_PIN A9  IOSTANDARD LVCMOS33 } [get_ports { io_i_uart_rx }];
6
7  # Mapeo de los 4 bits menos significativos del PC sobre 4 diodos LED monócromos
8  set_property -dict { PACKAGE_PIN H5  IOSTANDARD LVCMOS33 } [get_ports { io_pc_val[0] }];
9  set_property -dict { PACKAGE_PIN J5  IOSTANDARD LVCMOS33 } [get_ports { io_pc_val[1] }];
10 set_property -dict { PACKAGE_PIN T9  IOSTANDARD LVCMOS33 } [get_ports { io_pc_val[2] }];
11 set_property -dict { PACKAGE_PIN T10 IOSTANDARD LVCMOS33 } [get_ports { io_pc_val[3] }];
12
13 # Mapeo del del reloj y la señal de reset a los pines correspondientes en la placa
14 set_property -dict { PACKAGE_PIN E3  IOSTANDARD LVCMOS33 } [get_ports { clock }];
15 set_property -dict { PACKAGE_PIN C2  IOSTANDARD LVCMOS33 } [get_ports { reset }];
16
17 # Definición de la señal de reloj que empleará Vivado en el análisis
18 create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { clock }];

```

Figura 6.3: *Constraints* configurados para probar el diseño final en la placa.

El reloj de la placa de desarrollo funciona, por defecto, a una frecuencia de 100 MHz. A esa frecuencia de trabajo resulta muy complicado observar las actualizaciones del contador de programa que se reflejan en el array de LEDs, por lo que se ha implementado un divisor de frecuencia con el que ralentizar la ejecución de los programas en la CPU.

El código que implementa el mecanismo se muestra en la Figura 6.4 y su funcionamiento es como sigue:

Clase SlowTick

La clase SlowTick alberga un registro empleado como contador incremental. Este contador se incrementa en una unidad a cada ciclo, y se resetea al alcanzar el valor indicado por el parámetro *divisor*. Al resetearse, este módulo genera un pulso de un ciclo de duración en su única salida (valor 1 en el puerto *tick*).

Clase BUFGCE

El módulo BUFGCE no define ningún hardware en sí mismo, si no que constituye una referencia a un componente definido fuera del código, bien sea un hardware que no puede ser descrito en Chisel,

o un componente físico presente en el hardware donde se vaya a desplegar el diseño. Esto se consigue haciendo que el módulo extienda la clase `BlackBox`, con lo que se indica, precisamente, que el módulo no se encuentra definido en Chisel.

En este caso, el hardware al que se hace referencia es un buffer de reloj presente en la placa de desarrollo [28]. El buffer recibe como entradas una señal de reloj (`io.I`) y una señal de habilitación (`io.CE`) de tal modo que, siempre que la señal de habilitación esté activa, se deje pasar a través del buffer un pulso del reloj (puerto de salida `io.O`).

Lo que proporciona la clase definida en el código es una interfaz para poder interactuar con el buffer y, cuando Vivado reciba el código Verilog del diseño, reconocerá la instanciación del módulo `BUFGCE` como una primitiva propia de Xilinx, sustituyéndolo por el recurso físico correspondiente.

Modificaciones en el módulo Top

En primer lugar, se crean las instancias de los módulos `SlowTick` y `BUFGCE` (señales `slow` y `gate`, respectivamente). La entrada `io.I` del `BUFGCE` será la señal de reloj implícita de Chisel y, la señal de habilitación será el pulso generado por `SlowTick` cada `Parameters.ClockMult` ciclos.

Por último, se genera la instancia del módulo `CPU` indicando que la señal de reloj que gobierne su comportamiento no será la implícita de Chisel, si no la generada por el buffer de reloj.

Si por ejemplo se configurase `SlowTick` para generar el pulso cada 5×10^7 ciclos, la duración del ciclo observada por el computador sería de:

$$\underbrace{\frac{1}{100 \cdot 10^6}}_{\text{Período del reloj}} \cdot \underbrace{5 \cdot 10^7}_{\text{Número de ciclos}} = 0,5 \text{ s}$$

```

1  class SlowTick(divisor: Int) extends Module {
2      val io = IO(new Bundle {
3          val tick = Output(Bool())
4      })
5      withReset(~reset.asBool) {
6          val counter = RegInit(0.U(32.W))
7          counter := Mux(counter === (divisor-1).U, 0.U, counter + 1.U)
8          io.tick := (counter === 0.U)
9      }
10 }
11
12 class BUFGCE extends BlackBox(Map("CE_TYPE" -> StringParam("\\"SYNCH\\""))) {
13     val io = IO(new Bundle {
14         val I  = Input(Clock())
15         val CE = Input(Bool())
16         val O  = Output(Clock())
17     })
18 }
19
20 class Top extends Module {
21     ...
22     val slow = Module(new SlowTick(Parameters.ClockMult))
23     val gate = Module(new BUFGCE)
24     gate.io.I := clock
25     gate.io.CE := slow.io.tick
26     val clk_slow = gate.io.O
27     val cpu      = withClockAndReset(clk_slow, reset.asAsyncReset) { Module(new CPU) }
28     ...
29 }

```

Figura 6.4: Implementación de los módulos `SlowTick` y `BUFGCE`, y modificaciones realizadas en el módulo `Top`.

6.2. Análisis de los resultados temporales y de área obtenidos en Vivado

La integración del **BUFGCE** tenía el único propósito de hacer que la verificación de resultados de una prueba de concepto (carga y ejecución de un código sencillo), fuera algo realizable, pudiendo observar una actualización del contador de programa cada 0.5 segundos. No obstante, de cara a realizar un análisis de la arquitectura en condiciones normales de operación, se ha suprimido el **BUFGCE**, quedando en las *constraints*, tan solo, la asignación del reloj al pin correspondiente en la placa, y la definición del reloj `sys.clk_pin`.

A continuación se comentarán los resultados obtenidos por la herramienta Vivado tras la síntesis e implementación del diseño monociclo original y el diseño segmentado.

6.2.1. Análisis de la utilización

En la Figura 6.5 se muestra una tabla en la que se comparan los valores de utilización obtenidos con el comando `report_utilization -hierarchical` en la consola Tcl de Vivado. En primer lugar, se observa un decremento del 7% en el número de LUTs² ocupados por el núcleo (dato de la instancia 'cpu' en la columna 'Logic LUTs'). La incorporación de la *hazard unit* al pipeline, al ser esta una pieza de lógica combinacional tan sencilla, no ha supuesto un incremento en el área del núcleo. No obstante a esto, la memoria ocupa aproximadamente el 80% de la implementación (en términos de LUTs totales), por lo que, aún siendo algo singular, un decremento del 7% en el área del núcleo podría entrar dentro de lo esperado. Podría atribuirse, quizás, a las optimizaciones introducidas en el proceso de síntesis.

Un resultado esperado es el incremento del 30% observado en el número de Flip-Flops (columna 'FF'), como consecuencia directa de la introducción de los registros de segmentación.

CPU monociclo						
Instance	Module	Total LUTs	Logic LUTs	LUTRAMs	SRLs	FFs
Top	(top)	11457	3265	8192	0	1192
(Top)	(top)	0	0	0	0	0
cpu	CPU	1906	1906	0	0	1146
inst_fetch	InstrFetch	1226	1226	0	0	122
regs	RegisterFile	664	664	0	0	1024
mem	Memory	9464	1272	8192	0	0
uart	UART	87	87	0	0	46
uartRx	UartRx_mem	86	86	0	0	31

CPU segmentada						
Instance	Module	Total LUTs	Logic LUTs	LUTRAMs	SRLs	FFs
Top	(top)	11200	3008	8192	0	1589
(Top)	(top)	0	0	0	0	0
cpu	CPU	1771	1771	0	0	1543
regs	RegisterFile	577	577	0	0	992
srDE	DE	698	698	0	0	194
srEM	EM	381	381	0	0	108
srFD	FD_mod	49	49	0	0	98
srMW	MW	33	33	0	0	104
mem	Memory	9342	1150	8192	0	0
uart	UART	87	87	0	0	46
uartRx	UartRx_mem	85	85	0	0	31

Figura 6.5: Comparativa de los resultados de utilización obtenidos.

²*Look-Up Table*: el bloque básico de lógica combinacional de una FPGA. Estos bloques implementan por sí solos cualquier función lógica de un número determinado de entradas.

6.2.2. Análisis temporal

En el resumen del análisis temporal obtenido con el comando `report_timing_summary` se indican los siguientes valores para el WNS (*Worst Negative Slack*) de cada diseño:

- **Monociclo:** WNS (ns) = -19.983
- **Segmentado:** WNS (ns) = 0.058

El WNS es un valor que representa el margen temporal más desfavorable del diseño. En otras palabras, es la diferencia entre el tiempo de ciclo marcado como objetivo de la implementación (en este caso, 10 ns), y el tiempo que corresponde al camino crítico. Un valor negativo del WNS indica que existe al menos una ruta que no cumple con el tiempo de ciclo especificado.

A partir de las cifras obtenidas puede inferirse que la frecuencia máxima a la que puede trabajar el computador monociclo es de

$$F_{\max} = \frac{1}{T_{\text{crit}}} = \frac{1}{T_{\text{reloj}} - WNS} = \frac{1}{10 - (-19,983)} = 33,35 \text{ MHz}$$

donde T_{crit} es el tiempo que tarda una señal en recorrer el camino crítico, y T_{reloj} es el periodo de la señal de reloj ($1/f_{\text{reloj}}$). Por otro lado, al haber obtenido un slack positivo para el diseño segmentado, este podrá funcionar a los 100 MHz que marca el reloj de la placa.

Al ejecutar el comando `report_timing -max_paths 5` se obtiene un informe más detallado con los datos presentado en la Figura 6.6. Se observa el slack obtenido para cada implementación, el requisito temporal a cumplir (periodo del reloj en el apartado *Requirement*), el tiempo del camino crítico (dato del apartado *Data Path Delay*), y el número de niveles lógicos atravesados en ese camino crítico (*Logic Levels*).

CPU monociclo	
Slack (VIOLATED) :	-19.983ns (required time - arrival time)
Requirement:	10.000ns (sys_clk_pin rise@10.000ns - sys_clk_pin rise@0.000ns)
Data Path Delay:	29.924ns (logic 4.604ns (15.386%) route 25.320ns (84.614%))
Logic Levels:	24 (LUT2=1 LUT3=1 LUT5=2 LUT6=13 MUXF7=4 MUXF8=1 RAMD64E=2)
CPU segmentada	
Slack (MET) :	0.058ns (required time - arrival time)
Path Group:	sys_clk_pin
Path Type:	Setup (Max at Slow Process Corner)
Requirement:	10.000ns (sys_clk_pin rise@10.000ns - sys_clk_pin rise@0.000ns)
Data Path Delay:	9.583ns (logic 2.231ns (23.282%) route 7.352ns (76.718%))
Logic Levels:	11 (CARRY4=4 LUT3=1 LUT4=1 LUT6=5)

Figura 6.6: Selección de algunos de los valores obtenidos tras análisis temporal.

En resumen, gracias a la segmentación se ha conseguido reducir el tiempo del camino crítico en, aproximadamente, 20 ns, lo cual se traduce en una frecuencia de trabajo 3 veces superior o, lo que es lo mismo, se ha logrado un speedup de x3, alejado del x5 ideal, pero igualmente significativo.

7 Conclusiones y trabajo futuro

El objetivo principal del proyecto era la implementación, en el lenguaje de descripción de hardware Chisel, de un núcleo RISC-V segmentado capaz de procesar el conjunto no privilegiado de instrucciones base definido en la especificación RV32I del estándar.

Este trabajo tuvo como punto de partida un diseño monociclo incompleto, el cual se terminó de desarrollar como parte de un proceso de aprendizaje inicial. Esta fase de aprendizaje implicó, en primer lugar, la asimilación de los conceptos fundamentales inherentes al desarrollo en lenguaje Chisel. Además, se realizó un notorio ejercicio de indagación para comprender el funcionamiento interno del núcleo, las características internas y el conexionado de los módulos que lo integran, así como la metodología seguida en el proceso de pruebas tanto de componentes individuales, como de la arquitectura global.

Tras esta fase inicial, comenzó el proceso de elaboración de la arquitectura segmentada, el cual ha tenido como resultado dos implementaciones distintas: una segmentada en dos fases (*fetch*-resto), y la implementación final en cinco fases; siendo ambas el resultado de una metodología de pruebas basada en la ejecución de tests que cargasen en la memoria de la CPU códigos con los que generar los escenarios de especial interés que el núcleo debería resolver por medio de los mecanismos hardware implementados.

El empleo de GTKWave para el análisis de las trazas VCD generadas por los tests fue determinante en el proceso de depuración y validación del núcleo. La visualización detallada del comportamiento de la arquitectura en cada ciclo permitió detectar dependencias entre instrucciones, comprender su origen y verificar el correcto funcionamiento de los mecanismos implementados para su resolución, todo ello de una manera ágil y visual.

Por otro lado, Chisel ha demostrado ser el HDL intuitivo y eficaz que sus desarrolladores idearon. Su integración con el ecosistema de Scala y el empleo de conceptos propios de la programación orientada a objetos facilitan la organización modular del diseño, característica que ha permitido, sin duda, abordar la complejidad creciente de la arquitectura.

Como trabajo futuro podría plantearse, en primer lugar, la implementación del conjunto de extensiones contempladas en la extensión G (IMAFD, Zicsr y Zifencei), lo cual sentaría las bases para la ejecución de un sistema operativo embebido completo como, por ejemplo, FreeRTOS. Además, existen tests arquitecturales y funcionales de RISC-V con los que podría evaluarse el cumplimiento del estándar.

Por último, propondría evaluar el empleo de un flujo de diseño completamente abierto basado en el proyecto F4PGA¹, que aúna un conjunto de herramientas *open source* para la síntesis, *place and route* y generación del bitstream. De este modo podría construirse un flujo de desarrollo basado íntegramente en tecnologías abiertas (Chisel + RISC-V + F4PGA).

¹<https://f4pga.readthedocs.io/en/latest/getting-started.html>

Bibliografía

- [1] ARM Holdings plc. *Annual Report 2025 (Form 20-F)*. Inf. téc. Consultado el 14 de julio de 2026. Cap. Notes to Condensed Consolidated Financial Statements - Our business model, págs. 31-32. URL: <https://www.sec.gov/Archives/edgar/data/1973239/000197323925000024/arm-20250630.htm>.
- [2] OpenRISC Community. *Architecture*. Consultado el 28 de mayo de 2025. 2025. URL: <https://openrisc.io/architecture>.
- [3] Tony Chen y David A. Patterson. *RISC-V Geneology*. Inf. téc. UCB/EECS-2016-6. Accedido el 28 de mayo de 2025. University of California at Berkeley, 2016, pág. 2. URL: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-6.pdf>.
- [4] RISC-V International. *RISC-V Landscape: Members*. Consultado el 28 de mayo de 2025. 2025. URL: <https://riscv.landscap2.io/?group=members>.
- [5] European Commission. *Proposal for a Regulation of the European Parliament and of the Council on a framework of measures for strengthening the Union's semiconductor ecosystem, repealing Regulation (EU) 2023/1781 (Chips Act 2.0)*. Consultado el 15 de julio de 2026. 2026. URL: <https://ec.europa.eu/newsroom/dae/redirection/document/129104>.
- [6] John Cocke y Victoria Markstein. «The evolution of RISC technology at IBM». En: *IBM Journal of research and development* 34.1 (1990). Consultado el 23 de agosto de 2025, págs. 4-11. URL: <https://ieeexplore.ieee.org/document/5389855>.
- [7] Andrew S Tanenbaum. «Implications of structured programming for machine architecture». En: *Communications of the ACM* 21.3 (1978). Consultado el 30 de agosto de 2025, págs. 237-246. URL: <https://dl.acm.org/doi/abs/10.1145/359361.359454>.
- [8] Andrew Waterman et al. «The risc-v instruction set manual, volume i: Base user-level isa». En: *EECS Department, UC Berkeley, Tech. Rep. UCB/EECS-2011-62* 116.2011 (2011). Consultado el 28 de mayo de 2025, págs. 1-32. URL: https://docs.alexrp.com/riscv/riscv_unpriv_v1_0.pdf.
- [9] RISC-V International. *About RISC-V International*. Consultado el 30 de agosto de 2025. 2025. URL: <https://riscv.org/about/#history>.
- [10] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, pág. 9. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [11] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 15-17. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [12] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 27-35. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.

- [13] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 35-37. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [14] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 37-38. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [15] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 38-40. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [16] Kito Cheng y Jessica Clarke. *RISC-V ABIs Specification*. Ver. August 12, 2025-draft. Consultado el 31 de agosto de 2025. RISC-V International. 2025, pág. 6. URL: <https://github.com/riscv-non-isa/riscv-elf-psabi-doc>.
- [17] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 25-26. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [18] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 49-52. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [19] Andrew Waterman et al. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture (User-Level ISA)*. Versión 20250508, Consultado el 31 de agosto de 2025. RISC-V International, Unprivileged ISA Committee. Mayo de 2025, págs. 47-48. URL: <https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj70mv8tFtkp/view>.
- [20] Jonathan Bachrach et al. «Chisel: constructing hardware in a scala embedded language». En: *Proceedings of the 49th annual design automation conference*. Consultado el 11 de julio de 2025. 2012, págs. 1216-1225. URL: <https://dl.acm.org/doi/abs/10.1145/2228360.2228584>.
- [21] VirtusLab. *Scala Survey Results 2023*. Consultado el 11 de julio de 2025. 2023. URL: <https://scalasurvey2023.virtuslab.com/>.
- [22] RISC-V International. *riscv-gnu-toolchain*. 2020. URL: <https://github.com/riscv-collab/riscv-gnu-toolchain>.
- [23] «IEEE Standard Hardware Description Language Based on the Verilog(R) Hardware Description Language». En: *IEEE Std 1364-1995* (1996). Consultado el 13 de julio de 2025, págs. 207-218. DOI: 10.1109/IEEESTD.1996.81542.
- [24] AMD. *7 Series FPGAs Data Sheet: Overview*. 2020. URL: https://docs.amd.com/v/u/en-US/ds180_7Series_Overview.
- [25] Scott Chacon y Ben Straub. *Pro Git*. 2.^a ed. Consultado el 15 de julio de 2025. Berkeley, CA: Apress, 2014. ISBN: 978-1-4842-0076-6. URL: <https://git-scm.com/book/en/v2>.
- [26] Stack Exchange Inc. *2022 Developer Survey*. Consultado el 15 de julio de 2025. 2022. URL: <https://survey.stackoverflow.co/2022/#technology-version-control>.
- [27] LF Projects LLC. *Frequently Asked Questions - Get me Verilog*. Consultado el 21 de junio de 2026. 2020. URL: <https://www.chisel-lang.org/docs/resources/faqs#get-me-verilog>.
- [28] LF Projects LLC. *UltraScale Architecture Clocking Resources User Guide (UG572)*. Consultado el 9 de julio de 2026. 2025. URL: <https://docs.amd.com/r/en-US/ug572-ultrascale-clocking/BUFGCE-Clock-Buffers>.